



CDA STUDIO · RÉVISION DU TITRE PROFESSIONNEL

Comprendre les notions. Puis les expliquer.

Un cours progressif, des exemples autonomes
et des exercices pour vérifier ce que tu as compris.

Merise · TypeScript · SQL · Docker
Tests JavaScript · Backend · React

Édition révisée du 10 octobre 2026 · Exemples pédagogiques fictifs

Documentations officielles, spécifications et références académiques citées

À lire avec le recueil de cheatsheets pour le rappel rapide

Comprendre une étape à la fois

Une leçon commence par une idée, montre des données ou du code, puis explique comment on obtient le résultat. Les exemples sont fictifs : toutes les informations utiles sont données.

1. Lire le vocabulaire avant l'exemple

Quand un mot est nouveau, reformule sa définition. Par exemple, une cardinalité indique combien de liens un objet précis peut avoir. Lis ensuite la situation qui utilise ce mot.

2. Suivre les valeurs, pas seulement les lignes

Pour le code, note les valeurs de départ et ce qui change après chaque instruction. Pour une table, suis une ligne précise. Pour un schéma, lis chaque lien en faisant une phrase.

3. Chercher avant de lire le corrigé

Les exercices réutilisent une règle expliquée juste avant. Leur correction montre les étapes du raisonnement ; le lien du corrigé et le lien de retour permettent de passer de l'un à l'autre.

4. Utiliser le mémo après la leçon

La fiche mémo rappelle les règles et les syntaxes déjà expliquées. Cache-la ensuite et essaie de retrouver la règle avec un exemple.

5. Lire les bases avant l'approfondissement

Les passages marqués « Approfondissement » détaillent des cas particuliers. Termine d'abord les exemples de base, puis reviens sur ces passages quand tu sais expliquer le cas simple.

Les repères JavaScript qui suivent expliquent les symboles utilisés dans les chapitres. Le sommaire est cliquable ; les références documentaires sont regroupées à la fin.

Quelques repères pour lire le JavaScript

Les exemples du cours utilisent JavaScript et TypeScript. Tu peux suivre leur fonctionnement sur papier : commence par reconnaître les valeurs, puis les instructions qui les utilisent.

1. Une variable donne un nom à une valeur

Dans `const prix = 10;`, `prix` est le nom et `10` est la valeur. Le signe `=` affecte la valeur au nom : il ne pose pas une question. `const` signifie que ce nom ne sera pas réaffecté. Un texte se place entre guillemets, comme `"Livre"`. `true` et `false` sont des valeurs booléennes : vrai et faux.

2. Une fonction reçoit des valeurs et produit un résultat

`calculerTotal` est une fonction. Ses paramètres `prix` et `quantite` sont les noms utilisés à l'intérieur. Quand on l'appelle avec `calculerTotal(10, 2)`, ces paramètres valent `10` et `2`. `return` renvoie le résultat `20` à l'endroit où la fonction a été appelée. `console.log` affiche une valeur ; `//` commence un commentaire, sans effet sur le calcul.

Premier programme complet : calculer un total

```
const prix = 10;
const quantite = 2;
function calculerTotal(prix, quantite) {
  return prix * quantite;
}
const total = calculerTotal(prix, quantite);
console.log(total); // affiche 20
```

3. Une condition choisit ce qui sera exécuté

`if` signifie « si ». `===` compare deux valeurs sans les convertir : avec `total = 20`, `total === 20` est vrai. Le deuxième programme affiche donc `Montant correct`. Les accolades `{ }` délimitent un bloc d'instructions ; `else` contient le cas contraire. Dans les chapitres, `>=` signifie « supérieur ou égal » et `!` inverse un booléen.

Deuxième programme, indépendant du premier

```
const total = 20;
if (total === 20) {
  console.log("Montant correct");
} else {
  console.log("Autre montant");
}
```

Pour approfondir : [08] [07] [10] [09] · Références détaillées à la fin.

SUITE DE LA LEÇON

Quelques repères pour lire le JavaScript

4. Un objet regroupe des propriétés ; un tableau contient une liste

Un objet peut réunir l'identifiant et le titre d'un livre. Une propriété associe un nom à une valeur : titre vaut "Le Phare". Ici les accolades écrivent un objet ; dans if, elles délimitaient des instructions. Un tableau est une liste ordonnée : [10, 12] contient deux nombres. Le programme suivant transforme chaque note en son double.

Lire le point, les crochets et la flèche

livre.titre lit la propriété titre de l'objet livre. Les crochets [] délimitent ici un tableau. map fabrique un nouveau tableau en appliquant une fonction à chaque élément : 10 devient 20 et 12 devient 24. note => note * 2 est une fonction courte : elle reçoit note et renvoie son double. Une fonction fournie à une autre fonction est souvent appelée callback.

Troisième programme : objet, liste et transformation

```
const livre = { id: 1, titre: "Le Phare" };
console.log(livre.titre); // Le Phare
const notes = [10, 12];
const doubles = notes.map(note => note * 2);
console.log(doubles); // [20, 24]
```

5. Importer une fonction et lire un fragment

import sert à récupérer ce qu'un autre fichier ou une bibliothèque exporte. export rend une valeur disponible à d'autres fichiers. Une bibliothèque est du code réutilisable, par exemple React. Le cours indique si un bloc est un programme autonome ou un fragment : un fragment illustre une étape et suppose le contexte annoncé. Tu n'as pas à deviner des variables cachées.

Une manière de vérifier ta compréhension

Pour chaque exemple, indique une valeur de départ, suis les instructions dans l'ordre et note le résultat. Dans le premier programme, remplace la quantité 2 par 3 : le résultat devient 30. Si un symbole te bloque, reviens à son explication avant de passer à la suite.

Pour approfondir : [08] [07] [10] [09] · Références détaillées à la fin.

Choisir une matière

Dans chaque chapitre, les leçons suivent un ordre de lecture. Commence par la première si les bases sont encore floues.

01 · Merise et conception / page 6

Un cours progressif avec des données visibles, des lectures guidées et des exercices corrigés - 8 leçons et 3 fiches de rappel

02 · TypeScript · comprendre ce que le code promet / page 28

Progression : valeurs et types → objets et listes → entrées inconnues → promesses et fichiers → génériques. Prérequis : lire les exemples JavaScript de l'introduction. Les utilitaires sont un approfondissement.

03 · SQL et PostgreSQL · suivre les lignes pas à pas / page 40

Progression : tables et règles → lecture → jointure → regroupement → écriture et transaction. Prérequis : reconnaître un tableau de lignes et colonnes. EXISTS, paramètres et plans sont un approfondissement après ces bases.

04 · Docker : fabriquer, lancer et relier un programme / page 58

Distinguer ce qui est fabriqué, ce qui tourne et ce qui reste après l'arrêt.

05 · Tests JavaScript : partir d'une règle vérifiable / page 64

Préparer, appeler, comparer ; puis vérifier les échanges et nettoyer.

06 · Backend : comprendre le chemin d'une demande / page 70

Recevoir une requête, vérifier les données et les droits, puis accéder au stockage.

07 · React · comprendre comment les données deviennent un écran / page 85

Des composants aux URL, pas à pas. Prérequis : JavaScript de l'introduction.

1. Comprendre un besoin avant de dessiner

Nous allons concevoir une petite bibliothèque fictive. Commençons par ce que ses utilisateurs veulent faire, puis cherchons les règles et les informations nécessaires.

Notre situation de départ

Le catalogue contient deux livres : Le Phare et Les Jardins. Un livre désigne ici un ouvrage du catalogue. Un exemplaire est une copie physique de cet ouvrage : deux copies du Phare sont deux exemplaires du même livre. Un visiteur consulte le catalogue sans compte. Un lecteur inscrit emprunte des copies. Un administrateur ajoute des copies à la bibliothèque.

Dire à qui le résultat sert

Un acteur est un rôle dans l'utilisation du système. Une personne peut consulter le catalogue comme visiteur, puis utiliser son compte comme lecteur. Le rôle décrit ce qu'elle fait dans une situation ; il ne nous dit pas encore comment stocker ses données.

Formuler une user story

Une user story est une phrase qui exprime un besoin du point de vue d'un acteur. Sa formule est : « En tant que [acteur], je souhaite [action], afin de [bénéfice]. » L'action dit ce que la personne veut faire. Le bénéfice explique pourquoi cela lui est utile.

Trois histoires complètes

- En tant que visiteur, je souhaite rechercher un livre par son titre, afin de savoir si la bibliothèque le possède.
- En tant que lecteur inscrit, je souhaite voir mes prêts en cours, afin de connaître les copies à rendre.
- En tant qu'administrateur, je souhaite ajouter une copie à un livre du catalogue, afin de la rendre disponible au prêt.

Rendre le besoin vérifiable

Une phrase générale ne suffit pas pour décider si le résultat est correct. Un critère d'acceptation donne une situation précise et le résultat attendu. Pour la recherche, nous décidons que chercher un mot du titre affiche les titres contenant ce mot.

Pour approfondir : [01] [28] · Références détaillées à la fin.

SUITE DE LA LEÇON

1. Comprendre un besoin avant de dessiner

Données et résultats attendus

Situation	Résultat attendu
Titres : Le Phare et Les Jardins. Recherche : Phare.	Afficher Le Phare seulement.
Mêmes titres. Recherche : Montagne.	Afficher aucun livre et expliquer le résultat vide.
Lina possède deux prêts actifs et un prêt rendu.	Afficher ses deux prêts actifs.

Ajouter les règles de gestion

Une règle de gestion est une obligation du domaine étudié. Ici, chaque exemplaire appartient à un seul livre ; son numéro d'inventaire est unique ; il ne peut avoir qu'un prêt actif à la fois. Ces règles précisent ce que les histoires laissaient ouvert. Il faut les recueillir auprès des personnes concernées, plutôt que les deviner.

Relier ces outils à Merise

Merise est une méthode de conception des systèmes d'information. Dans ce cours, nous utiliserons surtout ses modèles de données pour représenter livres, copies et liens. Les user stories et les cas d'utilisation UML sont des outils complémentaires de description des besoins ; ils ne sont pas des modèles de données Merise. À l'étape suivante, nous nommerons les informations à conserver.

À TOI DE JOUER · MER-01

Nouvelle règle : un prêt peut être renouvelé une seule fois, pour sept jours. Un prêt a déjà été renouvelé et sa date de retour est le 20 juin. Donne le résultat attendu d'une nouvelle demande.

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [01] [28] · Références détaillées à la fin.

2. Passer des livres réels au MCD

Un modèle est une représentation simplifiée de la réalité. Le MCD, modèle conceptuel de données, représente les types d'objets et les liens que les règles nous demandent de conserver.

Observer avant de nommer

Notre catalogue contient le livre 7, intitulé Le Phare, et le livre 8, intitulé Les Jardins. Les copies physiques 10 et 11 appartiennent toutes les deux au livre 7. Le livre 8 n'a encore aucune copie. Il existe donc deux types d'objets distincts : les livres du catalogue et leurs exemplaires physiques.

Une entité et ses occurrences

Dans le vocabulaire Merise, un type d'objet est une entité. Nous appelons les deux entités LIVRE et EXEMPLAIRE. Un objet précis de ce type est une occurrence : le livre 7 est une occurrence de LIVRE ; la copie 10 est une occurrence d'EXEMPLAIRE. Écrire LIVRE dans le dessin ne signifie pas dessiner un seul livre.

Décrire et reconnaître chaque objet

Un attribut est une information portée par un objet, comme le titre d'un livre. Un identifiant est une information, ou un ensemble d'informations, permettant de reconnaître chaque occurrence sans ambiguïté. Nous choisissons livre_id pour les livres et exemplaire_id pour les copies. Deux livres pourraient porter le même titre : le titre ne sera donc pas notre identifiant.

Écrire le dictionnaire des données

Le dictionnaire explique le sens de chaque donnée. Son domaine indique les valeurs autorisées. Nous décidons ici que les identifiants sont des entiers positifs et que le titre est un texte non vide.

Informations à conserver

Nom	Sens	Valeurs autorisées
livre_id	Identifiant du livre dans le catalogue.	Entier positif et unique : 7, 8...
titre	Titre affiché dans le catalogue.	Texte non vide : Le Phare.
exemplaire_id	Identifiant d'une copie physique.	Entier positif et unique : 10, 11...

Nommer le lien

Une association représente un lien entre des entités. L'association « posséder » relie ici un livre et une copie. Pour le livre 7, nous observons deux liens : avec la copie 10 et avec la copie 11. Nous devons aussi représenter les cas autorisés demain, comme un livre sans copie.

Pour approfondir : [33] [23] · Références détaillées à la fin.

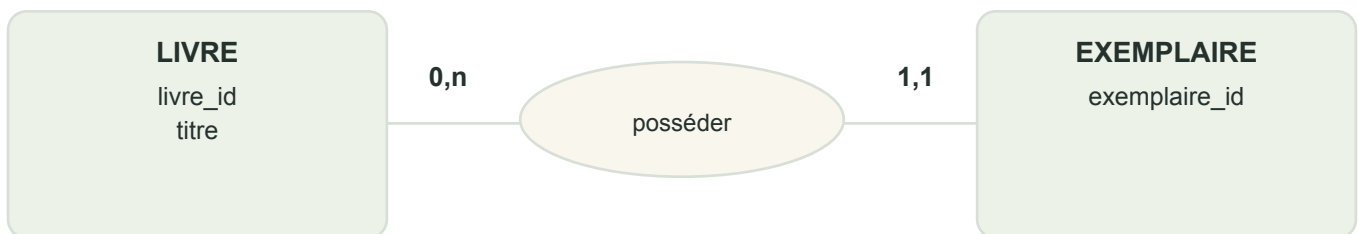
SUITE DE LA LEÇON

2. Passer des livres réels au MCD

Compter les liens depuis un seul objet

Une cardinalité comporte deux nombres : le minimum, puis le maximum de liens autorisés pour une occurrence. Près de LIVRE, demander : « Pour un livre précis, combien de copies ? » Réponse : zéro au minimum, plusieurs possibles au maximum, donc 0,n. Près d'EXEMPLAIRE, demander : « Pour une copie précise, combien de livres ? » Réponse : exactement un, donc 1,1. La lettre n signifie plusieurs possibles ; elle n'exige pas plusieurs liens présents.

LIVRE (0,n) - posséder - EXEMPLAIRE (1,1)



Un livre : de zéro à plusieurs exemplaires.

Un exemplaire : exactement un livre.

Les quatre lectures usuelles

Notation	Lecture depuis une occurrence
0,1	Zéro ou un lien. Exemple : une personne peut avoir au plus un dossier.
1,1	Exactement un lien. Exemple : chaque copie appartient à un livre.
0,n	Zéro, un ou plusieurs liens. Exemple : un livre peut avoir des copies.
1,n	Un ou plusieurs liens. Exemple : une commande validée doit avoir une ligne.

À TOI DE JOUER · MER-02

Avec les règles de cette leçon, le livre 8 peut-il exister sans copie ? La copie 12 peut-elle exister sans livre ? Justifie en lisant chaque côté du dessin.

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [33] [23] · Références détaillées à la fin.

3. Deux objets ou trois objets dans le même fait ?

Une association doit conserver le sens exact d'un lien. Compter ses participants aide à savoir si deux relations séparées expriment vraiment le même fait qu'un lien plus complet.

Binaire : deux types d'objets

Une association binaire met en relation deux types d'entités. Dans la bibliothèque, chaque lien « posséder » associe un livre précis à une copie précise. Le lien livre 7-copie 10 est déjà complet : aucun troisième objet n'est nécessaire pour dire à quel livre cette copie appartient.

Ternaire : trois types d'objets

Prenons un autre exemple, indépendant de la bibliothèque. Une école attribue un enseignant à une matière dans une classe. Dire « Alice enseigne SQL » ne précise pas à quelle classe. Dire « Alice enseigne à C2 » ne précise pas la matière. Le fait complet relie enseignant, matière et classe : c'est une association ternaire. Un triplet désigne ces trois valeurs prises ensemble.

Les trois affectations réellement enregistrées

Enseignant	Matière	Classe
Alice	SQL	C1
Alice	JavaScript	C2
Bob	SQL	C2

Essayer de remplacer les triplets par des paires

Une paire conserve seulement deux valeurs du triplet. En oubliant la classe, nous obtenons les enseignants et leurs matières. En oubliant la matière, nous obtenons les enseignants et leurs classes. En oubliant l'enseignant, nous obtenons les matières et leurs classes. Cette opération s'appelle une projection.

Ce qui reste dans chacune des trois listes

Liste de paires	Valeurs présentes
Enseignant-matière	Alice-SQL ; Alice-JavaScript ; Bob-SQL.
Enseignant-classe	Alice-C1 ; Alice-C2 ; Bob-C2.
Matière-classe	SQL-C1 ; JavaScript-C2 ; SQL-C2.

Pour approfondir : [33] [04] · Références détaillées à la fin.

SUITE DE LA LEÇON

3. Deux objets ou trois objets dans le même fait ?

Reconstituer un fait et chercher l'erreur

Essayons Alice-SQL-C2. Étape 1 : Alice-SQL existe dans la première liste. Étape 2 : Alice-C2 existe dans la deuxième. Étape 3 : SQL-C2 existe dans la troisième. Les trois vérifications passent. Pourtant, la table des affectations ne contient pas Alice-SQL-C2 : SQL en C2 est assuré par Bob. Les paires permettent donc d'inventer un fait.

Le triplet Alice-SQL-C2 n'existe pas, même si ses paires existent

Faits autorisés

Alice - SQL - C1
Alice - JavaScript - C2
Bob - SQL - C2

Trois listes de paires

Rejoindre les trois listes
peut inventer Alice - SQL - C2

Les trois paires existent ; le triplet Alice - SQL - C2 n'existe pas.
Conserver une affectation complète, ou justifier la décomposition.

Comprendre « sans perte »

Décomposer signifie séparer une relation en plusieurs relations. Une décomposition sans perte permet de reconstruire exactement les lignes de départ. Elle ne doit ni en perdre ni en inventer. Ici, la perte porte sur la combinaison précise des trois valeurs ; lors de la reconstruction, elle produit une ligne supplémentaire.

Conserver l'information utile

Pour notre école, nous gardons les trois valeurs ensemble dans une affectation. La leçon suivante expliquera comment traduire les liens en tables. Le choix entre binaire et ternaire vient du sens des faits et des règles : trois entités dans un domaine ne signifient pas automatiquement qu'une association ternaire est nécessaire.

À TOI DE JOUER · MER-03

En utilisant uniquement les trois listes de paires ci-dessus, Bob-JavaScript-C2 passerait-il les vérifications ? Compare avec Alice-SQL-C2.

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [33] [04] · Références détaillées à la fin.

4. Construire les tables : comprendre 1:n et n:n

Nous savons maintenant quels objets et liens existent. Le MLD, modèle logique de données, traduit ici ce dessin en tables avec des colonnes, des lignes et des références.

Lire une table et ses clés

Une table rassemble des objets ou des liens ayant les mêmes informations. Une ligne décrit un cas précis ; une colonne décrit une information. On appelle aussi relation l'ensemble des lignes dans le modèle relationnel. La clé primaire identifie chaque ligne sans doublon ni valeur absente. La clé étrangère conserve une référence vers une clé d'une autre table. Les abréviations sont PK pour clé primaire et FK pour clé étrangère.

Premier cas : un livre, plusieurs copies

Nous reprenons les règles connues : un livre peut avoir zéro ou plusieurs copies ; chaque copie appartient à exactement un livre. Les maximums sont un et plusieurs : on parle d'un lien 1:n. Cette notation abrégée décrit les maximums ; les minimums restent ceux du MCD.

Table LIVRE : une ligne par livre

livre_id	titre
7	Le Phare
8	Les Jardins

Table EXEMPLAIRE : une ligne par copie

exemplaire_id	livre_id
10	7
11	7

Pourquoi la référence se place dans EXEMPLAIRE

Chaque copie a un seul livre : une colonne livre_id suffit dans sa ligne. Les copies 10 et 11 portent la même référence 7, mais leurs propres identifiants restent différents. Le livre 8 existe sans copie. Une liste de copies dans une cellule de LIVRE compliquerait le stockage de plusieurs liens ; la table des copies les représente une ligne à la fois.

Pour approfondir : [23] [48] · Références détaillées à la fin.

SUITE DE LA LEÇON

4. Construire les tables : comprendre 1:n et n:n

Deuxième cas : plusieurs des deux côtés

Une boutique possède les commandes 100 et 101 et les produits 5, Stylo, et 6, Gomme. Une commande validée contient un ou plusieurs produits ; un produit peut apparaître dans zéro ou plusieurs commandes. Chaque produit apparaît au plus une fois par commande. Chaque présence porte une quantité entière positive. Les maximums sont plusieurs des deux côtés : c'est un lien n:n.

Les achats que nous devons représenter

Commande	Produit	Quantité
100	5 - Stylo	2
100	6 - Gomme	1
101	5 - Stylo	5

Créer une ligne pour chaque lien

Une seule référence dans COMMANDE ne permettrait pas ses deux produits. Une seule référence dans PRODUIT ne permettrait pas les deux commandes du Stylo. Nous créons donc LIGNE, une table de liaison. Ses trois lignes correspondent aux trois achats du tableau. commande_id référence COMMANDE ; produit_id référence PRODUIT. La quantité appartient à ce lien : le produit 5 est acheté en quantité 2 dans la commande 100 et en quantité 5 dans la commande 101.

Identifier un lien avec deux valeurs

Dans LIGNE, commande_id seul se répète et produit_id seul se répète. Leur couple identifie chaque ligne, puisque nous avons interdit deux lignes pour le même produit dans une commande. Une clé composée utilise ainsi plusieurs colonnes ensemble. Nous choisissons ce couple comme clé primaire.

MLD abrégé : les flèches indiquent les références

```
COMMANDE(commande_id PK)
PRODUIT(produit_id PK, libelle)
LIGNE(commande_id FK → COMMANDE,
       produit_id FK → PRODUIT, quantite)
PK de LIGNE = (commande_id, produit_id)
```

Pour approfondir : [23] [48] · Références détaillées à la fin.

SUITE DE LA LEÇON

4. Construire les tables : comprendre 1:n et n:n

MCD : sens ; MLD : tables et clés ; MPD : moteur précis

À TOI DE JOUER · MER-04

On ajoute le produit 6 en quantité 3 à la commande 101. Quelles valeurs place-t-on dans LIGNE ?
Peut-on identifier cette nouvelle ligne avec produit_id seul ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [23] [48] · Références détaillées à la fin.

5. Du lien 1:1 aux contraintes du MPD

Une table peut décrire le bon lien tout en acceptant des données interdites. Le MPD, modèle physique de données, choisit les types et contraintes qui feront respecter ces règles dans un moteur comme PostgreSQL.

Poser les règles du cas 1:1

Une personne peut avoir zéro ou un dossier. Chaque dossier appartient à exactement une personne. Les maximums sont un de chaque côté : c'est un lien 1:1. Les minimums diffèrent : le dossier est facultatif pour une personne, mais sa personne est obligatoire. Nous connaissons les personnes 1, Alice, et 2, Bilal. Alice possède le dossier 10 ; Bilal n'en possède pas.

Les personnes

personne_id	nom
1	Alice
2	Bilal

Le dossier présent

dossier_id	personne_id
10	1

Déduire trois contraintes, une par règle

Étape 1 : la référence `personne_id` dans `DOSSIER` doit désigner une personne existante ; elle devient une clé étrangère. Étape 2 : chaque dossier doit avoir une personne ; cette référence ne peut pas être absente. Étape 3 : une personne ne peut pas avoir deux dossiers ; sa référence ne doit pas se répéter dans `DOSSIER`. Ces trois obligations sont différentes : aucune ne remplace les deux autres.

Comprendre les mots SQL

SQL est le langage utilisé ici pour créer et manipuler les tables. `INTEGER` signifie que la colonne contient un entier. `NULL` représente une valeur absente. Les clauses suivantes ajoutent les obligations à la définition des colonnes ; une contrainte est une règle que la base vérifie lorsqu'on écrit des données.

Pour approfondir : [23] [48] · Références détaillées à la fin.

SUITE DE LA LEÇON

5. Du lien 1:1 aux contraintes du MPD

Traduction des trois obligations

Clause SQL	Lecture dans cet exemple
REFERENCES personne(id)	La personne indiquée doit exister.
NOT NULL	Chaque dossier doit indiquer une personne.
UNIQUE	Une même personne ne peut apparaître deux fois dans DOSSIER.
PRIMARY KEY	Cet identifiant distingue les lignes sans doublon ni absence.

Tables vides créées dans PostgreSQL

```
CREATE TABLE personne (  
    id INTEGER PRIMARY KEY  
);  
CREATE TABLE dossier (  
    id INTEGER PRIMARY KEY,  
    personne_id INTEGER NOT NULL UNIQUE  
        REFERENCES personne(id)  
);  
INSERT INTO personne VALUES (1), (2);  
INSERT INTO dossier VALUES (10, 1);
```

Prévoir le résultat d'une écriture

Un second dossier pour `personne_id = 1` est refusé par `UNIQUE`. Un dossier pour `personne_id = 3` est refusé par la clé étrangère : aucune personne 3 n'existe. Un dossier sans personne est refusé par `NOT NULL`. Un premier dossier pour la personne 2 serait accepté. La personne 2 peut aussi rester sans dossier, conformément à son minimum 0.

Traduire aussi le lien ternaire

Reprenons l'école : enseignants, matières et classes possèdent chacun un identifiant. Nous créons une table `AFFECTATION` avec trois références, une vers chaque table. Sa ligne `Alice-SQL-C1` conserve les trois participants ensemble. Dans cet exemple, seul un triplet complet ne peut pas se répéter ; aucune paire n'est déclarée unique. La clé primaire contient donc les trois références. Identifiants : Alice=1, Bob=2 ; SQL=1, JavaScript=2 ; C1=1, C2=2.

Pour approfondir : [23] [48] · Références détaillées à la fin.

SUITE DE LA LEÇON

5. Du lien 1:1 aux contraintes du MPD

Les trois lignes d'AFFECTATION

enseignant_id	matiere_id	classe_id
1	1	1
1	2	2
2	1	2

Ce que les clés ne suffisent pas à imposer

Une référence de LIGNE vers COMMANDE garantit que la commande existe. Elle ne garantit pas qu'une commande possède au moins une ligne. Ce minimum demande une vérification supplémentaire adaptée au fonctionnement de l'application. Pour une quantité positive, CHECK (quantite > 0) vérifie la condition ; NOT NULL reste nécessaire si l'absence est interdite. Les index facilitent certaines recherches ; ils sont aussi des choix du MPD.

À TOI DE JOUER · MER-05

Après les insertions du code, teste mentalement séparément les dossiers (11, 1), (12, 3) et (13, 2). Les valeurs sont (id, personne_id). Lesquels sont acceptés ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [23] [48] · Références détaillées à la fin.

6. Dépendances, clés et 1NF : partir des données

Pour choisir les bonnes tables, il faut comprendre quelles informations en déterminent d'autres. Nous allons lire cette idée dans un petit tableau avant d'utiliser les notations de normalisation.

Énoncer les règles de CLIENT

Chaque client possède un identifiant unique, un seul email unique et obligatoire, et un nom. Plusieurs clients peuvent avoir le même nom. Pour cette leçon, l'email peut changer, mais chaque état valide doit respecter son unicité. Voici deux clients différents qui s'appellent Lina.

Table CLIENT

client_id	email	nom
1	lina.a@example.test	Lina
2	lina.b@example.test	Lina

Lire « détermine » dans une ligne

Si je connais `client_id = 1`, je retrouve un seul client, donc un seul email et un seul nom. Le même identifiant ne peut pas désigner demain deux emails dans deux lignes du même état valide. On dit que `client_id` détermine email. Cette obligation s'appelle une dépendance fonctionnelle.

Comprendre la flèche

On écrit `client_id → email`, et on lit « `client_id` détermine email ». Plus généralement, $X \rightarrow Y$ signifie que deux lignes ayant les mêmes valeurs dans les colonnes X doivent avoir les mêmes valeurs dans les colonnes Y. La flèche indique une règle sur les valeurs ; elle ne désigne pas nécessairement une clé étrangère entre deux tables.

Vérifier aussi le sens inverse

`email → client_id` est vraie ici, parce que notre règle impose un email unique. `nom → client_id` est fausse : le nom Lina correspond aux clients 1 et 2. Dans un domaine autorisant plusieurs comptes avec le même email, la première conclusion serait différente. Une dépendance vient donc des règles, et un tableau sert à illustrer ou à contredire leur lecture.

Pour approfondir : [44] [04] · Références détaillées à la fin.

SUITE DE LA LEÇON

6. Dépendances, clés et 1NF : partir des données

Reconnaître les différentes clés

Une superclé est un ensemble de colonnes qui permet de distinguer chaque ligne. `client_id` suffit ; `client_id` accompagné du nom suffit aussi. Une clé candidate est une superclé minimale : si on enlève une de ses colonnes, elle ne suffit plus. `client_id` et `email` sont chacun une candidate ; le couple `client_id`-nom n'en est pas une, car `nom` est superflu. La clé primaire est la candidate choisie pour identifier officiellement les lignes.

1NF : une cellule correspond à une valeur du domaine

NF signifie forme normale. La première forme, 1NF, demande ici une valeur atomique par cellule : une valeur du domaine choisi, sans liste de valeurs ni groupe répété. Pour les téléphones, notre domaine est un seul numéro par champ. « 0601, 0602 » représente deux numéros, pas un numéro unique. Les numéros sont abrégés pour cet exemple.

Avant : une liste dans une cellule

client_id	nom	telephones
1	Lina	0601, 0602

Représenter chaque téléphone séparément

Nous gardons le client dans `CLIENT` et créons `TELEPHONE`. Chaque ligne de `TELEPHONE` contient une référence au client et un seul numéro. Le client 1 peut ainsi avoir deux lignes sans être dupliqué dans `CLIENT`. Le couple `client_id`-numero identifie ces lignes si un même numéro ne se répète pas pour un client.

Après : table TELEPHONE

client_id	numero
1	0601
1	0602

Pour approfondir : [44] [04] · Références détaillées à la fin.

SUITE DE LA LEÇON

6. Dépendances, clés et 1NF : partir des données

Pourquoi cette étape vient en premier

Avant d'étudier les dépendances partielles ou transitives, nous devons disposer de lignes dont les colonnes ont un sens stable. La suite partira de tables contenant déjà une valeur par cellule. Leur organisation pourra néanmoins répéter inutilement certaines informations.

À TOI DE JOUER · MER-06

Le client 3 existe dans CLIENT avec le nom Noé et l'email noe@example.test. Il possède un seul téléphone, 0603. Quelle ligne ajouter à TELEPHONE ? Son nom nouveau rend-il nom unique par règle ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [44] [04] · Références détaillées à la fin.

7. Normaliser avec des tables avant et après

Une table peut respecter 1NF et répéter des informations au mauvais endroit. Normaliser consiste à réorganiser ces informations pour que chaque changement se fasse à l'endroit qui lui correspond.

2NF : le problème du libellé répété

Dans notre boutique, un produit possède un seul libellé. Une commande contient au plus une ligne par produit. La seule clé candidate de LIGNE est le couple commande-produit ; sa quantité dépend de ce couple. Voici trois lignes, avec une valeur par cellule : la table respecte 1NF.

Avant : LIGNE

Commande	Produit	Libellé	Quantité
100	5	Stylo	2
100	6	Gomme	1
101	5	Stylo	5

Identifier la dépendance partielle

Le libellé Stylo apparaît deux fois. Le modifier dans une seule ligne produirait deux libellés pour le produit 5. La règle est produit → libellé. Elle utilise seulement une partie de la clé commande-produit : c'est une dépendance partielle. Nous séparons la description du produit et son achat dans une commande.

Après : PRODUIT

Produit	Libellé
5	Stylo
6	Gomme

Après : LIGNE

Commande	Produit	Quantité
100	5	2
100	6	1
101	5	5

Pour approfondir : [44] [04] · Références détaillées à la fin.

SUITE DE LA LEÇON

7. Normaliser avec des tables avant et après

Lire le résultat 2NF

Le produit 5 est décrit une seule fois. Les deux achats le référencent et gardent leurs quantités propres. La deuxième forme normale, 2NF, exige la 1NF et supprime les dépendances d'attributs n'appartenant à aucune clé candidate envers une partie stricte d'une candidate.

3NF : une information qui en détermine une autre

Chaque commande possède un seul client, et chaque client un seul nom. La clé de cette nouvelle table est Commande. Les règles sont commande → client et client → nom. Le nom dépend donc de la commande en passant par le client : cette dépendance est transitive.

Avant : COMMANDE

Commande	Client	Nom
100	1	Lina
101	1	Lina

Séparer pour modifier une seule fois

Si Lina change de nom, nous ne voulons pas corriger chaque commande. Nous conservons le nom dans CLIENT. COMMANDE garde seulement sa référence au client. La troisième forme normale, 3NF, traite notamment ces dépendances transitives entre attributs qui n'appartiennent à aucune clé candidate.

Après : CLIENT

Client	Nom
1	Lina

Après : COMMANDE

Commande	Client
100	1
101	1

Pour approfondir : [44] [04] · Références détaillées à la fin.

SUITE DE LA LEÇON

7. Normaliser avec des tables avant et après

Approfondissement : BCNF

La forme de Boyce-Codd, BCNF, est plus exigeante que 3NF. Prenons un autre cas : un professeur enseigne une seule matière ; pour un élève et une matière, on choisit un seul professeur. Les deux clés candidates sont élève-matière et élève-professeur.

Avant : affectations scolaires

Élève	Matière	Professeur
Lina	SQL	Sarah
Noé	SQL	Sarah
Lina	JavaScript	Julien

Comprendre la différence avec 3NF

Un attribut premier appartient à une clé candidate. Ici, les trois attributs sont premiers. La dépendance professeur → matière est admise par 3NF, puisque matière est première. BCNF exige que tout ensemble déterminant une information extérieure soit une superclé. Sarah apparaît pour deux élèves : professeur ne distingue pas les lignes. Cette dépendance viole donc BCNF.

Après : PROFESSEUR_MATIERE

Professeur	Matière
Sarah	SQL
Julien	JavaScript

Après : ELEVE_PROFESSEUR

Élève	Professeur
Lina	Sarah
Noé	Sarah
Lina	Julien

Pour approfondir : [44] [04] · Références détaillées à la fin.

SUITE DE LA LEÇON

7. Normaliser avec des tables avant et après

Deux vérifications après décomposition

Retrouver la matière de chaque professeur reconstitue les lignes initiales : la décomposition est sans perte. Mais ajouter Luc-SQL à la première table et Lina-Luc à la seconde donnerait deux professeurs de SQL à Lina. La règle élève-matière → professeur demande donc une vérification supplémentaire : toutes les dépendances ne sont pas conservées dans les tables séparées.

À TOI DE JOUER · MER-07

Dans la première table LIGNE, on ajoute seulement un identifiant artificiel à chaque ligne et on conserve l'unicité du couple commande-produit. Le libellé du produit 5 reste répété. A-t-on supprimé le problème de normalisation ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [44] [04] · Références détaillées à la fin.

8. Décrire un service avec UML : include et extend

Les modèles Merise de données répondent à « quelles informations et quels liens conserver ? ». Les cas d'utilisation UML répondent à « quel service le système rend-il à un acteur ? » : les deux outils se complètent.

Situer UML et le système étudié

UML est un langage de modélisation défini par l'OMG, un organisme de normalisation. Un cas d'utilisation décrit un service qui donne un résultat utile à un acteur extérieur au système. Prenons un distributeur bancaire. Le système étudié est le distributeur ; l'acteur est le client ; son objectif est de retirer de l'argent.

Fixer les règles de notre exemple

Une carte valide et un solde suffisant sont nécessaires au retrait. Le distributeur vérifie toujours le code fourni. Si le code est incorrect, il ne délivre aucun argent. Lors d'un retrait réussi, le client peut demander un reçu papier. Le retrait reste entièrement réalisé si le client ne demande pas ce reçu.

Écrire le déroulement avant les flèches

Une précondition décrit ce qui doit être vrai pour commencer un scénario. Nous supposons ici une carte valide et un solde suffisant. Le scénario normal décrit les étapes menant au résultat. Une alternative décrit ce qui se passe quand le déroulement change, par exemple un code incorrect.

Scénario normal du retrait

- Le client choisit le retrait et indique un montant.
- Le distributeur demande le code et le vérifie.
- Si les conditions sont satisfaites, il délivre l'argent.
- À la proposition de reçu, il imprime seulement si le client le demande.

include : réutiliser un comportement inclus

Le cas « Retirer de l'argent » exécute « Vérifier le code ». Nous pouvons représenter ce comportement séparément pour le réutiliser. Le lien include part du cas qui l'utilise vers le cas inclus : Retirer → Vérifier le code. Include signifie inclusion du comportement ; il ne désigne pas simplement l'étape suivante d'un dessin.

Pour approfondir : [47] · Références détaillées à la fin.

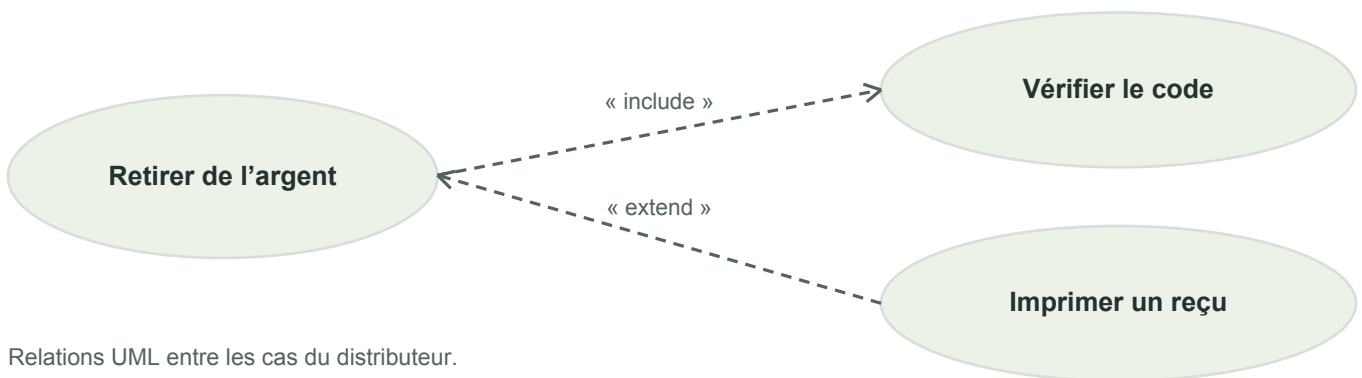
SUITE DE LA LEÇON

8. Décrire un service avec UML : include et extend

extend : ajouter un comportement sous condition

Le cas « Imprimer le reçu » ajoute un comportement au retrait, si le client le demande. Un point d'extension nomme l'endroit du cas de base où cet ajout peut intervenir : ici, la proposition de reçu après la délivrance de l'argent. Le lien extend part du comportement ajouté vers le cas complété : Imprimer le reçu → Retirer. Le retrait est le cas de base ; il est déjà complet sans cette extension.

Observer les deux directions de flèche



Relations UML entre les cas du distributeur.

Lire le dessin en deux phrases

Lien	Phrase complète
include	Retirer de l'argent utilise le comportement Vérifier le code.
extend	Imprimer le reçu complète Retirer de l'argent, si le client demande un reçu.

Tester une alternative

Avec un code incorrect, le résultat est un retrait refusé : aucun argent délivré. Le diagramme seul ne décrit pas tous ces détails. Il faut accompagner les ovals et les flèches de règles, de scénarios et de résultats. Un dessin de cas d'utilisation ne remplace pas cette description.

Pour approfondir : [47] · Références détaillées à la fin.

SUITE DE LA LEÇON

8. Décrire un service avec UML : include et extend

Revenir à la conception des données

Une user story explique un souhait et son bénéfice. Un cas d'utilisation développe un service et ses variantes. Le MCD Merise décrit ensuite les informations nécessaires au domaine. Le mot Client peut représenter un acteur dans UML et un type de donnée dans un modèle, mais le rôle d'acteur ne suffit pas à décider d'une nouvelle entité ou table.

À TOI DE JOUER · MER-08

La règle change : chaque retrait réussi imprime obligatoirement un reçu. Si on conserve un cas séparé « Imprimer le reçu », quel lien choisit-on avec « Retirer » ? Dans quel sens va la flèche ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [47] · Références détaillées à la fin.

1 · Un type décrit les valeurs autorisées

Objectif : lire une annotation de type et distinguer une erreur repérée avant l'exécution d'une erreur produite pendant l'exécution. Commencer par des valeurs simples.

Pourquoi ajouter des types ?

JavaScript sait manipuler nombres, textes et booléens. Un booléen vaut true ou false. TypeScript ajoute des descriptions pour vérifier certaines opérations avant de lancer le programme : c'est le contrôle statique. « Statique » signifie ici que l'outil relit le code sans l'exécuter. Le type number autorise des nombres ; string des textes ; boolean les deux booléens. Une annotation s'écrit après deux-points : quantité: number signifie « cette variable doit contenir un nombre ». Le signe = donne la valeur ; les deux-points décrivent sa forme. Ne pas confondre ces rôles.

Programme complet TypeScript · valeurs.ts

```
const quantite: number = 2;
const titre: string = 'LIRE';
const disponible: boolean = true;
console.log(quantite + 1);
console.log(titre.toUpperCase());
console.log(disponible);
// Affichage : 3, puis LIRE, puis true.
```

Relire chaque opération

quantite + 1 additionne deux nombres et produit 3. titre.toUpperCase() appelle une méthode du texte : elle produit une nouvelle chaîne en majuscules, LIRE. console.log affiche la valeur dans le terminal. Si on écrivait const quantite: number = "deux", TypeScript signalerait une incompatibilité : la valeur est un texte. Dans const quantite = 2, l'outil peut déduire le type depuis la valeur ; cette déduction s'appelle l'inférence. L'annotation est utile pour annoncer un contrat, mais elle n'est pas obligatoire partout.

Le contrôle ne s'exécute pas à ta place

Le navigateur ou Node exécute du JavaScript. Les annotations TypeScript sont retirées lorsque le code devient du JavaScript. Elles ne sont donc pas un contrôle automatique des données reçues à travers un formulaire ou un réseau. JSON est un format textuel de données ; un JSON reçu peut contenir une quantité négative ou un texte inattendu. Il faut vérifier ces valeurs pendant l'exécution, appelée runtime. Une erreur de type et une règle métier sont également différentes : -1 est bien un number, mais peut être une quantité interdite.

Pour approfondir : [71] [72] [79] [45] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Un type décrit les valeurs autorisées

Essai facultatif dans un dossier vide

Pour lire cette leçon, aucun outil n'est nécessaire. Pour exécuter le programme, il faut Node et npm installés. Enregistrer le code dans `valeurs.ts` et lancer les commandes ci-dessous dans son dossier. `tsc` est le vérificateur TypeScript ; `--noEmit` lui demande de ne produire aucun fichier JavaScript. `--strict` active une famille de contrôles plus exigeants, notamment autour des valeurs absentes. `tsx` lance le fichier TypeScript, mais ne remplace pas la vérification par `tsc`.

Terminal · trois étapes distinctes

```
npm init -y
npm install -D typescript tsx
npx tsc --strict --noEmit valeurs.ts
npx tsx valeurs.ts
```

À retenir

Un type annonce des possibilités. Un contrôle statique cherche des incohérences dans le code ; un contrôle à l'exécution vérifie des valeurs réellement reçues. Le réglage strict peut être conservé dans `compilerOptions.strict` d'un fichier `tsconfig.json`. Ce fichier configure un projet ; il n'est pas nécessaire pour les commandes explicites de cet essai.

À TOI DE JOUER · TYP-01

La règle métier impose une quantité entière de 1 à 5. Pour chaque valeur, distingue le type et la règle : 3, -1, "3". Les trois instructions envisagées sont `const quantité: number = valeur`. Pourquoi TypeScript ne suffit-il pas ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [71] [72] [79] [45] · Références détaillées à la fin.

2 · Décrire un objet, une liste et une position

Objectif : reconnaître la forme d'une donnée composée. Prérequis : distinguer une valeur de son type, comme dans la leçon précédente.

Un objet regroupe des propriétés

Un objet associe des noms à des valeurs : { titre: "Lire", pages: 120 }. titre et pages sont ses propriétés. Une interface donne un nom à cette forme pour la réutiliser. Dans interface Livre, les accolades contiennent la description, pas un livre réel. titre: string exige un titre textuel. pages?: number ajoute un point d'interrogation : la propriété pages peut être absente. Le point livre.titre permet ensuite de lire une propriété du véritable objet.

Programme complet · une description et deux livres

```
interface Livre {  
  titre: string;  
  pages?: number;  
}  
  
const premier: Livre = { titre: 'Lire', pages: 120 };  
const second: Livre = { titre: 'Marcher' };  
console.log(premier.titre);  
console.log(second.pages);  
// Résultats : Lire, puis undefined.
```

Comprendre la valeur absente

undefined signifie ici que la propriété n'a pas de valeur fournie. Lire second.pages est possible, mais effectuer directement second.pages + 1 ne serait pas sûr avec strict : on peut manquer de nombre. Il faut d'abord tester cette présence. Un alias de type, introduit par type, donne également un nom à une description : type LivreCourt = { titre: string }. Une interface sert notamment aux formes d'objets ; type peut aussi nommer les unions et les tuples que nous allons lire. Dans ces exemples, aucun des deux ne fabrique de valeur à l'exécution.

Liste ou tuple : choisir le contrat

Une liste contient un nombre variable d'éléments d'un type : string[] se lit « tableau de textes ». Les crochets peuvent donc décrire un type, ou, ailleurs, construire une valeur telle que ["Lire", "Marcher"]. Un tuple fixe des positions et leur type : [string, number] décrit ici un texte en position 0 puis un nombre en position 1. Les indices commencent à 0. Le mot readonly interdit de modifier un tableau à travers cette référence dans le code vérifié ; il ne gèle pas la valeur JavaScript à l'exécution.

Pour approfondir : [72] [77] [76] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Décrire un objet, une liste et une position

Programme complet · lire une liste et un tuple

```
const titres: readonly string[] = ['Lire', 'Marcher'];
type Position = [string, number];
const emplacement: Position = ['rayon', 2];
console.log(titres[0]);
console.log(emplacement[0], emplacement[1]);
// Résultats : Lire, puis rayon 2.
// titres.push('Dormir') serait une erreur TypeScript.
```

Pourquoi plusieurs descriptions ?

La liste répond à « quels éléments puis-je stocker ? » ; le tuple répond à « que signifie chaque position ? ». Pour une donnée métier comportant plusieurs champs, un objet nommé est souvent plus lisible qu'un long tuple. Le type `readonly` empêche ici `push`, qui ajouterait un élément. Un autre accès au même tableau JavaScript pourrait cependant le modifier si son type le permet ; ne pas annoncer `readonly` comme une protection de sécurité.

Petites lectures à pratiquer

- `Livre` décrit une forme ; premier est une valeur qui respecte cette forme.
- `pages?` permet l'absence. Une valeur présente doit quand même être un nombre.
- `string[]` décrit les éléments ; `[string, number]` décrit deux positions.
- `readonly` impose une règle au vérificateur, pas un gel automatique.

À TOI DE JOUER · TYP-02

Règle donnée : une position contient le nom du rayon, puis son numéro. Parmi `["A", 4]`, `[4, "A"]` et `["A"]`, quelles valeurs respectent `[string, number]` ? Pour le livre `{ titre: "Lire" }`, quelle valeur lit pages ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [72] [77] [76] · Références détaillées à la fin.

3 · Vérifier une valeur inconnue avant de l'utiliser

Objectif : accepter une entrée dont la forme est incertaine, puis autoriser seulement les opérations justifiées. Prérequis : objets, propriétés et valeurs absentes.

Pourquoi unknown ?

Supposons qu'une fonction reçoive une valeur provenant d'ailleurs. Elle pourrait être un texte, un nombre ou null. L'annotation unknown signifie « je ne connais pas encore son type ». TypeScript refuse donc des opérations qui supposent un texte, comme `value.trim()`. `trim` enlève les espaces aux extrémités d'un texte. On doit apporter une preuve avant de l'appeler. `typeof value === "string"` compare le type observé au mot "string". Dans la branche où le test est vrai, l'outil sait que `value` est un texte : il a réduit les possibilités. Cette réduction s'appelle *narrowing*.

Programme complet · une vérification avant trim

```
function nettoyer(value: unknown): string {
  if (typeof value === 'string') {
    return value.trim();
  }
  return 'Texte requis';
}
console.log(nettoyer(' Alice '));
console.log(nettoyer(12));
// Résultats : Alice, puis Texte requis.
```

Deux moyens de contourner une preuve

any demande à l'outil de laisser passer de nombreuses opérations sans contrôle : une erreur reste possible à l'exécution. Une assertion telle que `value as string` annonce « traite cette valeur comme un texte ». Le mot `as` ne transforme pas 12 en "12" et ne vérifie aucun champ. Si l'hypothèse est fausse, `trim` peut échouer. Préférer un test réellement exécuté. Pour un objet inconnu, vérifier qu'il n'est pas null, puis ses propriétés, leurs types et enfin les règles métier. `typeof null` vaut "object" : ce seul test d'objet ne suffit pas.

Une union annonce plusieurs variantes

Le symbole `|` signifie « ou ». `number | null` autorise un nombre ou une absence volontaire. Une union d'objets peut également représenter succès ou erreur. Les textes fixes "ok" et "error" sont ici des valeurs autorisées précises, pas n'importe quel texte. Le champ `status` permet de reconnaître la variante : c'est un discriminant. On doit le tester avant de lire le champ réservé à cette variante.

Pour approfondir : [76] [72] [73] · Références détaillées à la fin.

SUITE DE LA LEÇON

3 · Vérifier une valeur inconnue avant de l'utiliser

Programme complet · choisir une branche du résultat

```
type Resultat =  
  | { status: 'ok'; nom: string }  
  | { status: 'error'; message: string };  
function recevoir(): Resultat {  
  return { status: 'ok', nom: 'Alice' };  
}  
const resultat = recevoir();  
if (resultat.status === 'ok') {  
  console.log(resultat.nom);  
} else {  
  console.log(resultat.message);  
}  
// Affichage : Alice.
```

Relier chaque test à une opération

Dans la variante ok, le nom existe. Dans la variante error, c'est message qui existe. Un objet ayant seulement status: "ok" ne respecte pas Resultat : nom est obligatoire. Cela rend les cas à traiter visibles dans le code. Une autre vérification utile est error instanceof Error : elle demande si une valeur correspond à un objet Error avant de lire son message. Nous l'utiliserons pour une erreur asynchrone. Aucun de ces types ne dispense de valider des données extérieures.

À TOI DE JOUER · TYP-03

La fonction nettoyer ci-dessus reçoit " Bilal ", puis null. Prédisez les deux affichages. Autre résultat fourni : { status: "error", message: "Nom absent" }. Quel champ peut-on lire après avoir vérifié status === "error" ? Pourquoi écrire as string ne remplace-t-il pas le test ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [76] [72] [73] · Références détaillées à la fin.

4 · Attendre un résultat et partager du code entre fichiers

Objectif : lire une Promise et comprendre import/export. Prérequis : une fonction reçoit des arguments, retourne une valeur et peut signaler une erreur.

Une Promise n'est pas encore son résultat

Une opération asynchrone peut fournir son résultat plus tard. Une Promise représente cet engagement : elle pourra réussir avec une valeur, ou échouer avec une erreur. `Promise<string>` se lit « promesse dont le succès fournit un texte ». Une fonction portant `async` retourne toujours une Promise, même si son `return` contient directement un texte. La durée n'est pas garantie : `async` n'impose pas un délai et ne rend pas tous les calculs parallèles.

Programme complet · fournir un nom ou une erreur

```
async function chargerNom(id: number): Promise<string> {
  if (id === 1) {
    return 'Alice';
  }
  throw new Error('Utilisateur absent');
}
```

await et try/catch

`await` attend la réussite de la Promise à cet endroit de la fonction `async`. Il permet de travailler avec le texte obtenu plutôt qu'avec la Promise. Si la Promise échoue, le contrôle passe à `catch` lorsqu'un `try/catch` l'entoure. `throw` signale une erreur ; `new Error` construit un objet d'erreur. Avec `strict`, la valeur reçue par `catch` est inconnue : une vérification `instanceof Error` donne accès à `message`. Une Promise pourrait être rejetée avec une autre valeur ; ce test garde donc son utilité.

À ajouter sous `chargerNom` dans le même fichier

```
async function afficher(id: number): Promise<void> {
  try {
    const nom = await chargerNom(id);
    console.log(nom);
  } catch (error) {
    if (error instanceof Error) {
      console.log(error.message);
    }
  }
}
afficher(1); // Affichage : Alice.
```

Pour approfondir : [73] [76] [75] · Références détaillées à la fin.

SUITE DE LA LEÇON

4 · Attendre un résultat et partager du code entre fichiers

Suivre la valeur dans les trois étapes

`chargerNom(1)` produit une `Promise<string>`. `await chargerNom(1)` donne ensuite le texte Alice. `console.log` l'affiche. `Promise<void>` annonce que `afficher` ne fournit pas de donnée de succès à utiliser : son but est l'affichage. Remplacer l'appel final par `afficher(9)` conduit au message Utilisateur absent. Ici la fonction de chargement est fournie dans le cours ; aucune API réseau ou base personnelle n'est nécessaire. Pour l'essayer, réunir `chargerNom` et `afficher` dans `promesses.ts` : `npx tsc --strict --target ES2022 --noEmit promesses.ts` vérifie cet usage de Promise, puis `npx tsx promesses.ts` l'exécute.

Un module est un fichier qui partage des éléments

`export` rend un élément disponible depuis un fichier ; `import` le récupère dans un autre. Dans `export type`, on partage seulement une description TypeScript ; `import type` la récupère, et cet import disparaît à l'exécution. Les accolades de l'import nomment exactement ce qu'on récupère. Le chemin `./` commence dans le dossier du fichier appelant. Les deux fragments suivants expliquent ce découpage ; ils ne constituent pas une procédure de configuration de projet.

Fragment · `outils.ts`, à convertir en `outils.js` par TypeScript

```
export type Utilisateur = { id: number; nom: string };
export function saluer(nom: string): string {
  return 'Bonjour ' + nom;
}
```

Fragment · `principal.ts`, avec le module précédent disponible

```
import { saluer } from './outils.js';
import type { Utilisateur } from './outils.js';
const personne: Utilisateur = { id: 1, nom: 'Alice' };
console.log(saluer(personne.nom));
// Affichage attendu : Bonjour Alice.
```

Pour approfondir : [73] [76] [75] · Références détaillées à la fin.

SUITE DE LA LEÇON

4 · Attendre un résultat et partager du code entre fichiers

Ce qu'il manque pour exécuter ces modules

Ces derniers fragments supposent un outil TypeScript configuré pour convertir les fichiers .ts en modules JavaScript exécutables ; outils.js désigne alors le fichier produit depuis outils.ts. Cette configuration dépend de l'environnement. Pour cette leçon conceptuelle, suivre les valeurs suffit : le type Utilisateur aide à vérifier personne ; la fonction saluer existe réellement dans le JavaScript obtenu. Ne pas confondre import d'un type et validation d'un objet reçu.

À TOI DE JOUER · TYP-04

Deux appels indépendants sont donnés : chargerNom(1) et chargerNom(9). Quel est leur type avant await ? Que devient chacun dans un try/catch avec await ? Dans principal.ts, quel import apporte du comportement à l'exécution ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [73] [76] [75] · Références détaillées à la fin.

5 · Garder une relation de types avec un générique

Objectif : comprendre pourquoi une fonction réutilisable doit conserver le type de ses entrées.

Prérequis : listes, fonctions et valeurs absentes. Les utilitaires de fin de leçon sont un approfondissement.

Le problème : réutiliser sans perdre une information

On veut récupérer le premier élément d'une liste. Avec une liste de nombres, le résultat sera un nombre ; avec une liste de textes, un texte. Décrire tout avec `any` ferait perdre cette relation au vérificateur. Un générique introduit un nom pour un type variable. Dans `<T>`, `T` est ce nom : il ne représente pas une valeur JavaScript. `T[]` se lit « liste d'éléments de type `T` ». Le résultat `T | undefined` annonce « même type que les éléments, ou absence si la liste est vide ». `readonly T[]` ajoute que la fonction ne peut pas modifier cette liste via ce type ; cela ne gèle pas l'objet JavaScript à l'exécution.

Programme complet · une fonction, trois cas

```
function premier<T>(valeurs: readonly T[]): T | undefined {
  return valeurs[0];
}
const nombre = premier([4, 8]);
const texte = premier(['Lire', 'Marcher']);
const vide = premier<number>([]);
console.log(nombre, texte, vide);
// Résultat : 4 Lire undefined.
```

Lire les substitutions une par une

Avec `[4, 8]`, TypeScript déduit `T = number` ; `nombre` a donc le type `number | undefined`. Avec `["Lire", "Marcher"]`, il déduit `string`. Dans `premier<number>([])`, les chevrons apportent explicitement le type quand la liste vide ne contient aucune valeur pour le déduire. La fonction s'exécute de la même manière dans chaque cas : lire la position 0. Le générique ne fabrique pas trois fonctions ni un contrôle à l'exécution. Il conserve une relation utile au contrôle statique.

Pour approfondir : [\[74\]](#) [\[78\]](#) [\[77\]](#) [\[72\]](#) · Références détaillées à la fin.

SUITE DE LA LEÇON

5 · Garder une relation de types avec un générique

Approfondissement : transformer une description

Les utilitaires sont des types déjà fournis par TypeScript. Ils construisent une autre description depuis un type existant ; ils ne modifient pas les objets. Supposons Utilisateur avec id et nom obligatoires. Un écran de modification partielle peut fournir seulement les champs à changer : `Partial<Utilisateur>` rend les deux propriétés facultatives. Un affichage qui ne montre que le nom peut utiliser `Pick<Utilisateur, "nom">`. Le mot `Pick` signifie « sélectionner ». Lire le résultat concret est plus utile que mémoriser uniquement le nom de l'outil.

Programme complet · utilitaires sur un objet défini

```
type Utilisateur = { id: number; nom: string };
const modification: Partial<Utilisateur> = { nom: 'Bilal' };
const resume: Pick<Utilisateur, 'nom'> = { nom: 'Alice' };
const sansId: Omit<Utilisateur, 'id'> = { nom: 'Alice' };
const fixe: Readonly<Utilisateur> = { id: 1, nom: 'Alice' };
console.log(modification.nom, resume.nom);
// Résultat : Bilal Alice.
```

Lire les autres transformations depuis cet objet

Écriture	Forme obtenue ou restriction
<code>Partial<Utilisateur></code>	id et nom deviennent facultatifs.
<code>Pick<Utilisateur, 'nom'></code>	Il reste seulement nom obligatoire.
<code>Omit<Utilisateur, 'id'></code>	id est retiré ; nom reste obligatoire.
<code>Readonly<Utilisateur></code>	Réaffecter id ou nom via ce type est interdit.
<code>Record<string, number></code>	Des clés textuelles sont associées à des nombres.

Programme complet · un dictionnaire typé

```
const stock: Record<string, number> = {
  Lire: 2,
  Marcher: 0
};
console.log(stock.Lire);
// Résultat : 2.
```

Pour approfondir : [74] [78] [77] [72] · Références détaillées à la fin.

SUITE DE LA LEÇON

5 · Garder une relation de types avec un générique

Les limites à conserver

Readonly ne gèle pas profondément des objets imbriqués et ne protège pas un objet reçu. Partial permet seulement l'absence de champs : un nom présent reste un texte. Omit enlève un champ de la description, sans effacer ce champ d'un objet JavaScript déjà existant. Record<string, number> n'assure pas qu'une clé arbitraire existe réellement : la lecture d'une clé manquante vaut undefined à l'exécution. Tous ces outils prolongent les types ; ils ne remplacent ni les règles métier ni la validation runtime.

À TOI DE JOUER · TYP-05

La liste donnée est ["Alice", "Bilal"]. Quel T est déduit par premier et quel résultat concret obtient-on ? Ensuite, Utilisateur = { id: number; nom: string }. { id: 2 } convient-il à Partial<Utilisateur> ? { nom: 2 } convient-il ? Explique les deux décisions.

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [74] [78] [77] [72] · Références détaillées à la fin.

1 · Une table conserve des faits et impose des règles

Objectif : comprendre une ligne, une colonne, une clé primaire et une clé étrangère. Le cas complet est une petite boutique : Alice a deux commandes ; Bilal n'en a aucune.

Donner un sens aux colonnes

SQL, Structured Query Language, est un langage pour interroger et modifier une base de données relationnelle. Une table contient des lignes de même forme ; une colonne décrit une information. client contient id, l'identifiant, et nom, le prénom affiché. commande contient son propre id, client_id, le client qui a commandé, et total, le montant. client_id = 1 signifie « cette commande appartient au client dont l'id vaut 1 », donc Alice. Le nom de la colonne est une convention ; c'est la contrainte de référence qui impose le lien.

Table client · mêmes données dans tout le chapitre

id	nom
1	Alice
2	Bilal

Table commande · deux achats d'Alice, aucun de Bilal

id	client_id	total
10	1	30
11	1	20

Identifier sans confondre

Une clé primaire, PRIMARY KEY ou PK, identifie une ligne : elle doit être présente et unique. Deux clients ne peuvent donc pas porter le même id ; deux commandes non plus. Une clé étrangère, FOREIGN KEY ou FK, garantit une référence existante lorsqu'une valeur est fournie. Ici client_id référence client(id). Elle refuse une commande pour le client 3, car seuls 1 et 2 existent. Plusieurs commandes peuvent référencer 1 : une clé étrangère n'impose pas l'unicité des références.

Pour approfondir : [48] [60] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Une table conserve des faits et impose des règles

Une valeur manquante n'est pas un zéro

NULL représente une valeur non renseignée ou inconnue. Ce n'est ni 0 ni un texte vide. NOT NULL rend un champ obligatoire. UNIQUE interdit les valeurs dupliquées selon la contrainte ; par défaut dans PostgreSQL, plusieurs NULL ne sont pas considérés comme un même doublon. CHECK ajoute une condition, par exemple total >= 0. Une condition CHECK qui devient inconnue à cause de NULL n'est pas refusée : ajouter NOT NULL impose aussi la présence du total.

Programme SQL, étape 1 · créer les deux tables

```
CREATE TEMP TABLE client (  
    id INTEGER PRIMARY KEY,  
    nom TEXT NOT NULL  
);  
CREATE TEMP TABLE commande (  
    id INTEGER PRIMARY KEY,  
    client_id INTEGER NOT NULL REFERENCES client(id),  
    total NUMERIC NOT NULL CHECK (total >= 0)  
);
```

Lire la création

INTEGER stocke un entier, TEXT un texte et NUMERIC un nombre décimal adapté ici au montant. CREATE TABLE crée la structure ; TEMP limite ces tables à la session ouverte. Les parenthèses contiennent les définitions de colonnes ; la virgule les sépare ; le point-virgule termine une instruction. REFERENCES client(id) déclare la clé étrangère directement sur client_id. Cette notation courte a le même rôle qu'une contrainte FOREIGN KEY écrite séparément.

Même connexion, étape 2 · remplir les tables

```
INSERT INTO client (id, nom)  
VALUES (1, 'Alice'), (2, 'Bilal');  
INSERT INTO commande (id, client_id, total)  
VALUES (10, 1, 30), (11, 1, 20);
```

Pour approfondir : [48] [60] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Une table conserve des faits et impose des règles

Ce que cet exemple demande

Prérequis pour exécuter : une connexion PostgreSQL de test disponible. Lancer création puis remplissage dans cette même connexion ; TEMP disparaît quand la session se ferme. INSERT INTO choisit la table et l'ordre des colonnes ; VALUES fournit les valeurs de chaque ligne dans ce même ordre. Les textes sont entre apostrophes. Pour apprendre le raisonnement sans installer PostgreSQL, les tableaux ci-dessus donnent toutes les données utilisées.

À TOI DE JOUER · SQL-01

Prédise chaque ajout à partir du même état initial, sans modifier les données : clients (1, Alice), (2, Bilal) et commandes 10/11 ci-dessus. Les ajouts envisagés sont (12, 3, 15), (12, 2, -1), (12, NULL, 15), (12, 2, 0). Quelles tentatives sont acceptées ? Chaque tuple suit id, client_id, total.

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [48] [60] · Références détaillées à la fin.

2 · Lire des lignes, puis filtrer, trier et limiter

Objectif : suivre une sélection depuis les données jusqu'au résultat. Prérequis : comprendre les colonnes de commande et les données de la leçon précédente.

La question à résoudre

Nous voulons afficher la commande située en deuxième position parmi celles de total au moins 20, en commençant par le plus grand total. Les données complètes sont les deux lignes ci-dessous. id est le numéro de commande, pas le numéro du client. Une requête SELECT ne modifie pas ces données : elle calcule un résultat. Les requêtes sont des fragments : elles supposent une connexion PostgreSQL de test et les tables créées dans la leçon 1. Pour prévoir leur résultat sur papier, les tableaux de cette leçon suffisent.

Table commande · deux achats d'Alice, aucun de Bilal

id	client_id	total
10	1	30
11	1	20

Nommer les étapes avant la syntaxe

FROM indique la table qui fournit les lignes. WHERE garde les lignes dont la condition est vraie ; >= signifie « supérieur ou égal ». SELECT choisit les colonnes à montrer. ORDER BY donne leur ordre ; DESC signifie décroissant, du plus grand au plus petit ; ASC signifie croissant et correspond à l'ordre par défaut. LIMIT fixe le nombre maximal de lignes à rendre ; OFFSET ignore les premières lignes du résultat ordonné. Nous allons ignorer une ligne, puis en conserver une.

Fragment · lecture de commande avec un ordre défini

```
SELECT id, total
FROM commande
WHERE total >= 20
ORDER BY total DESC, id ASC
LIMIT 1 OFFSET 1;
```

Pour approfondir : [61] [55] [54] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Lire des lignes, puis filtrer, trier et limiter

Dérouler cette requête sur les deux lignes

Étape	Lignes ou valeurs qui restent
FROM commande	Commande 10 : 30 ; commande 11 : 20.
WHERE total >= 20	Les deux passent : 30 >= 20 et 20 >= 20.
SELECT id, total	On garde ces deux colonnes pour l'affichage.
ORDER BY total DESC, id ASC	Ordre : (10, 30), puis (11, 20).
OFFSET 1	Ignorer la première : il reste (11, 20).
LIMIT 1	Conserver au maximum une ligne : (11, 20).

Ordre écrit et raisonnement

L'instruction commence par SELECT, mais pour comprendre ces clauses simples, partir de la table et du filtre aide à suivre les lignes. Ce déroulé est une explication du résultat, pas une promesse sur l'ordre physique des calculs du moteur. Le moteur peut choisir une autre méthode qui produit le même résultat. Dans ORDER BY, id ASC sert à départager deux commandes de même total. Une pagination sans ordre défini ne promet pas quelles lignes constitueront la page.

Fragment · demander seulement les montants plus grands

```
SELECT id, total
FROM commande
WHERE total >= 25
ORDER BY id;
```

Le deuxième résultat, sans saut

La commande 10 passe car 30 >= 25. La commande 11 ne passe pas car 20 < 25. Le résultat est donc une ligne, id 10 et total 30. Sans WHERE, les deux seraient conservées. Sans LIMIT/OFFSET, aucune ligne n'est supprimée pour la pagination. Si aucun total ne satisfait le filtre, le résultat est un ensemble de zéro ligne ; il n'est pas une erreur SQL.

Pour approfondir : [61] [55] [54] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Lire des lignes, puis filtrer, trier et limiter

S'entraîner à lire la demande

- Quelles colonnes faut-il montrer ? C'est SELECT.
- Dans quelle table sont les lignes ? C'est FROM.
- Quelles lignes satisfont la règle ? C'est WHERE.
- Quel ordre et quelle portion faut-il rendre ? ORDER BY, LIMIT, OFFSET.

À TOI DE JOUER · SQL-02

Données indépendantes : commande (10, client 1, total 30) et (11, client 1, total 20). Prédise `SELECT id, total FROM commande WHERE total >= 20 ORDER BY total ASC LIMIT 1`. Ensuite, ajoute `OFFSET 1` à cette requête : que devient le résultat ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [61] [55] [54] · Références détaillées à la fin.

3 · Joindre des tables et repérer une absence

Objectif : reconstruire les liens entre clients et commandes, puis comprendre la ligne conservée pour un client sans commande. Prérequis : SELECT, FROM et le lien client_id.

Pourquoi une jointure ?

client donne le nom ; commande donne le montant. Une jointure rapproche les lignes qui décrivent le même fait. La règle de rapprochement est commande.client_id = client.id. Le point choisit la colonne d'une table. Un alias est un nom court pour cette table dans la requête : client c permet d'écrire c.id ; commande co permet co.client_id. Ces alias ne renommeront pas les tables stockées. Les requêtes sont des fragments : elles supposent une connexion PostgreSQL de test et les tables créées dans la leçon 1. Pour prévoir leur résultat sur papier, les tableaux de cette leçon suffisent.

Table client · mêmes données dans tout le chapitre

id	nom
1	Alice
2	Bilal

Table commande · deux achats d'Alice, aucun de Bilal

id	client_id	total
10	1	30
11	1	20

Deux choix avant le code

INNER JOIN ne conserve que les correspondances : Alice avec commande 10 et Alice avec commande 11. Bilal disparaît car aucune commande n'a client_id = 2. LEFT JOIN conserve également chaque ligne de la table de gauche sans correspondance ; la table de gauche est ici client. Les colonnes de commande valent alors NULL pour Bilal. ON donne la condition de rapprochement, et non un filtre portant sur tous les clients.

Fragment · conserver même les clients sans commande

```
SELECT c.nom, co.id AS commande_id, co.total
FROM client c
LEFT JOIN commande co ON co.client_id = c.id
ORDER BY c.id, co.id;
```

Pour approfondir : [59] [50] · Références détaillées à la fin.

SUITE DE LA LEÇON

3 · Joindre des tables et repérer une absence

Résultat complet du LEFT JOIN

nom	commande_id	total
Alice	10	30
Alice	11	20
Bilal	NULL	NULL

Construire chaque ligne du résultat

Pour `c.id = 1`, le moteur trouve deux commandes liées à Alice ; il produit donc deux lignes. Pour `c.id = 2`, il n'en trouve aucune. `LEFT JOIN` garde quand même Bilal et fournit des colonnes de commande absentes. `AS commande_id` donne un nom clair à la colonne `co.id` dans le résultat. `ORDER BY c.id`, `co.id` rend d'abord les clients dans leur ordre, puis leurs commandes. Changer `LEFT` en `INNER` retire seulement la ligne de Bilal dans cet exemple.

Tester l'absence avec IS NULL

`NULL` signifie qu'aucune valeur n'est connue ici ; on ne sait pas si elle serait égale à un nombre donné. Une comparaison ordinaire avec `NULL` donne un résultat inconnu, pas vrai. `WHERE` ne conserve que vrai. Pour chercher l'absence, utiliser `IS NULL` ; pour chercher une valeur renseignée, `IS NOT NULL`. Puisque l'id d'une vraie commande est obligatoire, `co.id IS NULL` permet ici d'identifier un client sans correspondance.

Fragment · clients sans commande, sur les mêmes données

```
SELECT c.nom
FROM client c
LEFT JOIN commande co ON co.client_id = c.id
WHERE co.id IS NULL;
```

Pour approfondir : [59] [50] · Références détaillées à la fin.

SUITE DE LA LEÇON

3 · Joindre des tables et repérer une absence

Résultat de cette dernière lecture

Les deux lignes d'Alice ont co.id égal à 10 ou 11 : elles sont écartées. La ligne de Bilal a co.id NULL : elle est conservée. Le résultat contient seulement Bilal. Le filtre co.total > 0 aurait au contraire écarté Bilal, car la comparaison avec son total absent n'est pas vraie. Garder une ligne grâce à LEFT JOIN n'oblige donc pas WHERE à la conserver ensuite.

À TOI DE JOUER · SQL-03

Données complètes : clients 1/Alice et 2/Bilal ; commandes 10/client 1/30 et 11/client 1/20. Combien de lignes produit INNER JOIN sur client_id = id ? Combien produit LEFT JOIN ? Dans le LEFT JOIN, qui garde WHERE co.id IS NULL ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [59] [50] · Références détaillées à la fin.

4 · Compter et regrouper sans compter une absence

Objectif : obtenir une ligne de résumé par client. Prérequis : savoir expliquer les trois lignes du LEFT JOIN Alice/Alice/Bilal de la leçon précédente.

De trois lignes à deux groupes

Nous voulons le nombre de commandes et le montant total de chaque client, y compris Bilal sans commande. Une fonction d'agrégation résume plusieurs lignes : COUNT compte et SUM additionne. GROUP BY indique quelles colonnes définissent un groupe. Ici le client est identifié par c.id, et c.nom doit accompagner cet identifiant dans la lecture. Les données source ci-dessous sont complètes. Les requêtes sont des fragments : elles supposent une connexion PostgreSQL de test et les tables créées dans la leçon 1. Pour prévoir leur résultat sur papier, les tableaux de cette leçon suffisent.

Table client · mêmes données dans tout le chapitre

id	nom
1	Alice
2	Bilal

Table commande · deux achats d'Alice, aucun de Bilal

id	client_id	total
10	1	30
11	1	20

Lignes avant de les regrouper

client	co.id	co.total
Alice	10	30
Alice	11	20
Bilal	NULL	NULL

Pour approfondir : [57] [51] [59] [50] · Références détaillées à la fin.

SUITE DE LA LEÇON

4 · Compter et regrouper sans compter une absence

Que faut-il compter ?

COUNT(*) compte toutes les lignes du groupe, même la ligne conservée pour Bilal. COUNT(co.id) compte seulement les valeurs de co.id non NULL. Les id d'Alice sont 10 et 11 : le compte vaut 2. Pour Bilal, aucun id de commande n'est renseigné : le compte vaut 0. SUM(co.total) donne 30 + 20 = 50 pour Alice. Sans valeur à additionner pour Bilal, SUM donne NULL. COALESCE(SUM(...), 0) choisit la première valeur non NULL : il affiche donc 0 à la place de cette somme absente.

Fragment · compter, additionner et grouper

```
SELECT c.nom, COUNT(co.id) AS commandes,
       COALESCE(SUM(co.total), 0) AS total
FROM client c
LEFT JOIN commande co ON co.client_id = c.id
GROUP BY c.id, c.nom
ORDER BY c.id;
```

Résultat attendu après les calculs

nom	commandes	total
Alice	2	50
Bilal	0	0

Pourquoi ne pas grouper uniquement par prénom ?

Deux clients différents peuvent porter le même nom ; le prénom ne doit pas fusionner leurs commandes. L'identifiant distingue leurs groupes. Dans cette requête, les colonnes affichées sont soit des colonnes de groupement, soit un résultat d'agrégation. co.id brut ne pourrait pas être ajouté à SELECT sans décider lequel montrer dans le groupe d'Alice : il y en a deux. Le résumé a volontairement changé l'échelle des lignes.

WHERE et HAVING ne décident pas au même moment

WHERE filtre les lignes avant le résumé ; HAVING filtre les groupes calculés. Pour demander « clients ayant au moins deux commandes », compter d'abord, puis tester ce nombre avec HAVING COUNT(co.id) >= 2. Ajouter cette clause après GROUP BY et avant ORDER BY. Alice est conservée ; Bilal est écarté. Mettre WHERE co.total >= 25 avant le regroupement réduirait d'abord Alice à sa commande 10 ; le compte deviendrait 1. Ces deux questions ne sont pas équivalentes.

Pour approfondir : [57] [51] [59] [50] · Références détaillées à la fin.

SUITE DE LA LEÇON

4 · Compter et regrouper sans compter une absence

Lire la transformation

- Rapprocher les clients et commandes avec LEFT JOIN.
- Conserver ou écarter les lignes avec WHERE si demandé.
- Former un groupe par client, puis calculer COUNT et SUM.
- Décider quels groupes garder avec HAVING, puis ordonner le résultat.

À TOI DE JOUER · SQL-04

Les seules données sont Alice avec commandes de 30 et 20, et Bilal sans commande. 1) Que donne COUNT(*) pour le groupe Bilal après LEFT JOIN ? 2) Que donne COUNT(co.id) ? 3) Avec HAVING COUNT(co.id) >= 2, qui reste et avec quel total ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [57] [51] [59] [50] · Références détaillées à la fin.

5 · Modifier des lignes et décider de valider ou annuler

Objectif : comprendre INSERT, UPDATE, DELETE et le rôle d'une transaction. Prérequis : les deux clients et les deux commandes du chapitre, avec leurs contraintes.

Quatre actions différentes

CRUD est l'abréviation anglaise de Create, Read, Update, Delete : créer, lire, modifier et supprimer. En SQL, on retrouve INSERT, SELECT, UPDATE et DELETE. INSERT ajoute une ligne ; UPDATE change les colonnes de lignes existantes ; DELETE retire des lignes. Dans UPDATE et DELETE, WHERE décide quelles lignes sont visées. Sans WHERE, toutes les lignes de la table peuvent être touchées. La table complète de cet exemple contient seulement les deux commandes ci-dessous.

Table commande · deux achats d'Alice, aucun de Bilal

id	client_id	total
10	1	30
11	1	20

Une transaction regroupe les écritures

Une transaction est un ensemble d'opérations validées ou annulées ensemble. BEGIN commence ce groupe. COMMIT conserve ses changements ; ROLLBACK les annule. On dit que ces opérations sont atomiques : le groupe n'est pas validé à moitié. Cela ne signifie pas que toute concurrence disparaît ; la base applique aussi des règles d'isolation. Pour cette leçon, nous allons ajouter une commande, modifier un montant, supprimer une commande, observer, puis tout annuler. Les requêtes sont des fragments : elles supposent une connexion PostgreSQL de test et les tables créées dans la leçon 1. Pour prévoir leur résultat sur papier, les tableaux de cette leçon suffisent.

Fragment · un essai entièrement annulé

```
BEGIN;
INSERT INTO commande (id, client_id, total)
VALUES (12, 2, 15) RETURNING id;
UPDATE commande SET total = 40 WHERE id = 10;
DELETE FROM commande WHERE id = 11;
SELECT id, client_id, total FROM commande ORDER BY id;
ROLLBACK;
```

Pour approfondir : [60] [63] [58] [62] [49] · Références détaillées à la fin.

SUITE DE LA LEÇON

5 · Modifier des lignes et décider de valider ou annuler

Lire les trois écritures

INSERT ajoute la commande 12 de Bilal ; RETURNING id renvoie l'identifiant de la ligne ajoutée, donc 12. Cela n'exige pas un identifiant automatiquement généré : ici nous l'avons fourni. UPDATE commande SET total = 40 WHERE id = 10 ne change que le montant de la commande 10. DELETE ... WHERE id = 11 retire la commande 11. À l'intérieur de cette transaction, SELECT permet de regarder les changements déjà effectués par cette connexion.

Résultat du SELECT avant ROLLBACK

id	client_id	total
10	1	40
12	2	15

État après ROLLBACK, revenu au début

id	client_id	total
10	1	30
11	1	20

Pourquoi cette annulation est utile

La nouvelle commande 12 disparaît, la commande 11 revient, et le montant de 10 redevient 30. Remplacer ROLLBACK par COMMIT aurait conservé les lignes 10/40 et 12/15. Une transaction est utile lorsque plusieurs changements doivent former un seul résultat cohérent. Les contraintes continuent de s'appliquer : la commande 12 avec client_id 3 serait refusée. Après une erreur SQL dans une transaction PostgreSQL, il faut traiter l'erreur et annuler ce groupe avant de poursuivre normalement.

Pour approfondir : [60] [63] [58] [62] [49] · Références détaillées à la fin.

SUITE DE LA LEÇON

5 · Modifier des lignes et décider de valider ou annuler

Avant une modification réelle

Prévoir les lignes ciblées avec un SELECT portant le même WHERE aide à vérifier son intention. DELETE sur un identifiant absent supprime zéro ligne ; ce seul fait n'est pas automatiquement une erreur SQL. Pour connaître les lignes modifiées ou supprimées, RETURNING peut aussi être utilisé avec UPDATE ou DELETE. Une règle métier peut ensuite exiger exactement une ligne et traiter autrement un résultat vide.

À TOI DE JOUER · SQL-05

État initial complet : commande 10/client 1/30 et commande 11/client 1/20. Dans une transaction : UPDATE commande SET total = 99 WHERE id = 10, puis ROLLBACK. Quel montant garde 10 ? Si l'on utilisait COMMIT ? Combien de lignes viserait UPDATE commande SET total = 99 sans WHERE ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [60] [63] [58] [62] [49] · Références détaillées à la fin.

6 · Approfondissement : existence, paramètres et plan

Objectif : expliquer trois outils sans les confondre : EXISTS choisit des lignes, un paramètre sépare une valeur du SQL, un index peut améliorer un accès. Prérequis : lectures, relations et WHERE.

Question 1 : un client possède-t-il au moins une commande ?

Nous cherchons seulement les noms des clients ayant commandé, une seule fois par client. Les données complètes restent clients 1/Alice et 2/Bilal ; commandes 10/client 1/30 et 11/client 1/20. EXISTS répond vrai si la sous-requête trouve au moins une ligne. Une sous-requête est une requête placée à l'intérieur d'une autre. Ici elle utilise l'id du client examiné par la requête extérieure. Les requêtes sont des fragments : elles supposent une connexion PostgreSQL de test et les tables créées dans la leçon 1. Pour prévoir leur résultat sur papier, les tableaux de cette leçon suffisent.

Fragment · chaque client est testé séparément

```
SELECT c.nom
FROM client c
WHERE EXISTS (
  SELECT 1
  FROM commande co
  WHERE co.client_id = c.id
);
```

Suivre le test pour Alice puis Bilal

Pour Alice, c.id vaut 1 : la sous-requête trouve les commandes 10 et 11. EXISTS devient vrai et Alice apparaît une fois, car la requête extérieure parcourt client, pas les commandes. Pour Bilal, c.id vaut 2 : aucune ligne n'est trouvée ; il est écarté. SELECT 1 fournit une valeur quelconque pour les lignes trouvées : ce qui compte pour EXISTS est leur existence, pas le montant choisi. Une jointure brute aurait répété Alice.

Question 2 : où placer une valeur reçue ?

Assembler du SQL avec un texte saisi peut mélanger une donnée et des instructions : c'est le risque d'injection SQL. Un paramètre réserve une place à la valeur ; PostgreSQL emploie ici \$1 pour la première. La valeur est fournie séparément, sans fabriquer le texte de la requête. PREPARE nomme une requête préparée sur le serveur et précise le type du paramètre ; EXECUTE lui fournit sa valeur. Ce petit exemple reste du SQL, sans bibliothèque JavaScript implicite.

Pour approfondir : [52] [56] [53] [64] · Références détaillées à la fin.

SUITE DE LA LEÇON

6 · Approfondissement : existence, paramètres et plan

Fragment PostgreSQL · préparer puis lire la commande 10

```
PREPARE lire_commande(INTEGER) AS
SELECT id, total FROM commande WHERE id = $1;
EXECUTE lire_commande(10);
-- Résultat : id 10, total 30.
```

Séparation et validation sont deux tâches

Un pilote applicatif effectue cette séparation avec son API, par exemple une requête contenant \$1 et une liste de valeurs [10]. Il faut utiliser l'API réelle du pilote, pas remplacer \$1 soi-même avec une concaténation. Un paramètre représente une valeur, pas un nom de table ou de colonne. Pour un tri dynamique, choisir les noms possibles dans une liste autorisée. Vérifier en plus le domaine métier d'un identifiant reste nécessaire.

Question 3 : comment le moteur accède-t-il aux lignes ?

Un index est une structure auxiliaire qui peut faciliter certaines recherches ou certains tris. Il consomme de l'espace et demande une mise à jour lors des écritures. Il ne garantit pas qu'une requête sera plus rapide : sur deux lignes, lire toute la table peut être moins coûteux. EXPLAIN montre le plan estimé choisi par le moteur ; EXPLAIN ANALYZE exécute réellement la requête et ajoute des mesures.

Fragments · observer avant de conclure

```
CREATE INDEX commande_client_idx ON commande(client_id);
EXPLAIN SELECT id FROM commande WHERE client_id = 1;
EXPLAIN ANALYZE SELECT id FROM commande WHERE client_id = 1;
```

Pour approfondir : [52] [56] [53] [64] · Références détaillées à la fin.

SUITE DE LA LEÇON

6 · Approfondissement : existence, paramètres et plan

Ce qu'on peut prédire et ce qu'on doit mesurer

La lecture vise ici les commandes 10 et 11. Le choix exact du plan dépend des données, des statistiques et du moteur ; nous ne prétendons pas prédire une ligne d'EXPLAIN précise. ANALYZE exécute aussi une écriture si on lui donne UPDATE ou DELETE : le changement n'est pas fictif. Chercher un gain demande donc une mesure pertinente sur un jeu représentatif, pas seulement la présence d'un index.

À TOI DE JOUER · SQL-06

Données complètes : clients 1/Alice et 2/Bilal ; commandes 10 et 11 appartenant à 1. Que rend la requête EXISTS ? Que donne EXECUTE lire_commande(99) si 99 est absent ? Un index impose-t-il que PostgreSQL l'utilise ? EXPLAIN ANALYZE exécute-t-il un DELETE ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [52] [56] [53] [64] · Références détaillées à la fin.

1 - Une image devient un conteneur en cours d'exécution

Situation : un petit serveur doit répondre dans un navigateur. Docker permet de préparer son environnement puis de lancer le programme dans cet environnement.

1. Nommer les deux objets

Une image est un paquet préparé contenant le programme, ses fichiers et son environnement. Elle existe même quand rien ne tourne. Un conteneur est une exécution de cette image, avec ses processus, son réseau et ses fichiers modifiables. Arrêter un conteneur n'efface pas l'image. Deux conteneurs issus d'une image sont deux exécutions distinctes ; leurs fichiers ne sont pas automatiquement partagés.

2. Préparer un programme très simple

Pour pratiquer : Docker installé et démarré, Internet disponible, port 8080 libre. Dans un dossier vide, créer `api.mjs` avec le premier bloc. Il utilise Node, qui exécute JavaScript, sans bibliothèque à installer. `createServer` prépare un serveur ; `res` envoie la réponse. Celle-ci est du JSON, un format de données contenant ici la clé `ok` et la valeur `true`.

Fichier `api.mjs` : copier uniquement ces lignes

```
import { createServer } from "node:http";
createServer((req, res) => {
  res.setHeader("Content-Type", "application/json");
  res.end('{ "ok": true }');
}).listen(3000, "0.0.0.0");
```

3. Décrire la fabrication

Créer `Dockerfile` dans le même dossier. `FROM` fournit un environnement contenant Node : `node:22-alpine` est l'image choisie. `WORKDIR` fixe le dossier de travail ; `COPY` y place notre fichier. `RUN` agit pendant la fabrication : ici, vérifier la syntaxe. `CMD` indique le programme à lancer plus tard, au démarrage d'un conteneur. `RUN` et `CMD` n'agissent pas au même moment.

Fichier `Dockerfile` : copier uniquement ces lignes

```
FROM node:22-alpine
WORKDIR /app
COPY api.mjs .
RUN node --check api.mjs
CMD ["node", "api.mjs"]
```

Pour approfondir : [\[17\]](#) [\[16\]](#) [\[19\]](#) [\[15\]](#) · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Une image devient un conteneur en cours d'exécution

Terminal ouvert dans le dossier des deux fichiers

```
docker build -t atelier-api .  
docker run --name atelier-api -p 127.0.0.1:8080:3000 atelier-api
```

4. Comprendre les ports et observer

build fabrique l'image ; -t la nomme atelier-api ; le point désigne le dossier utilisé. run crée et démarre un conteneur nommé atelier-api. Un port est un numéro où un programme écoute les connexions. Notre serveur écoute 3000 dans le conteneur. 0.0.0.0 le rend joignable sur les interfaces réseau de ce conteneur ; cela ne publie pas le port. -p relie 8080 sur notre ordinateur à 3000 dans le conteneur ; 127.0.0.1 limite cet accès à notre ordinateur. Ouvrir `http://127.0.0.1:8080` : {"ok":true} est attendu. EXPOSE 3000 documenterait le port sans le publier.

5. Arrêter et reconstruire sans confusion

- `docker stop atelier-api` arrête le programme ; `docker start atelier-api` relance ce même conteneur. run crée un nouveau conteneur : un nom déjà utilisé provoque une erreur.
- Après modification d'api.mjs, suivre les commandes ci-dessous : reconstruire l'image, arrêter puis retirer l'ancien conteneur, et en créer un nouveau. start relancerait toujours l'ancien conteneur.
- Le cache réutilise des étapes de fabrication inchangées. Avec des dépendances npm, copier leurs fichiers de description avant les sources évite de les réinstaller à chaque modification.
- `.dockerignore` est un fichier qui exclut des fichiers du dossier envoyé à Docker. Y exclure les secrets et les fichiers inutiles ; ne pas incorporer un mot de passe dans l'image.

Après une modification : remplacer l'ancien conteneur

```
docker build -t atelier-api .  
docker stop atelier-api  
docker rm atelier-api  
docker run --name atelier-api -p 127.0.0.1:8080:3000 atelier-api
```

Pour approfondir : [17] [16] [19] [15] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Une image devient un conteneur en cours d'exécution

L'idée à expliquer en une phrase

L'image est ce que je prépare ; le conteneur est ce que je lance ; la publication du port permet à mon navigateur de joindre ce programme.

À TOI DE JOUER · DOC-01

Le serveur fonctionne dans le conteneur, mais `run` a été exécuté sans `-p`. Pourquoi le navigateur ne le joint-il pas sur 8080 ? Arrêter le conteneur détruit-il l'image ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [17] [16] [19] [15] · Références détaillées à la fin.

2 · Relier deux services et conserver les données

Situation : nous voulons lancer notre serveur et PostgreSQL, un programme qui stocke des données en tables. Ils doivent se joindre et les données doivent survivre au remplacement du conteneur de base.

1. Pourquoi utiliser Compose ?

Compose décrit plusieurs services dans `compose.yaml`, puis les lance ensemble. Un service est une partie de l'application : `api` pour le serveur, `db` pour la base. Compose crée par défaut un réseau commun : les services y communiquent par leur nom. Ce réseau interne et la publication d'un port vers notre ordinateur sont deux choses distinctes.

2. Préparer l'exercice

Utiliser l'image `atelier-api` de la leçon précédente. Arrêter son ancien conteneur avec `docker stop atelier-api` pour libérer 8080. Copier le bloc suivant dans `compose.yaml`, dans le même dossier, en gardant l'indentation. Notre serveur répond `{"ok":true}` mais ne lit pas encore la base : cet exercice prépare et explique leur réseau.

Fichier `compose.yaml` complet pour cet exercice

```
services:
  api:
    image: atelier-api
    ports: ["127.0.0.1:8080:3000"]
  db:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD: exemple-local
    ports: ["127.0.0.1:15432:5432"]
    volumes: ["records:/var/lib/postgresql/data"]
volumes:
  records:
```

3. Lire une adresse depuis le bon endroit

`localhost` signifie « cet ordinateur ou cet environnement réseau ». Dans le navigateur, il désigne notre ordinateur ; dans `api`, il désigne `api`, pas `db`. PostgreSQL écoute 5432 dans `db` : un futur code de lecture dans `api` utiliserait `db:5432`. L'outil SQL sur notre ordinateur utiliserait `localhost:15432`, l'accès publié. Publier 15432 n'est pas nécessaire à la communication interne `api` → `db`.

Pour approfondir : [12] [14] [13] [18] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Relier deux services et conserver les données

Trois appels, trois adresses

Depuis	Vers
Navigateur sur l'ordinateur	Serveur : http://localhost:8080
Programme dans api	Base : db, port 5432
Outil SQL sur l'ordinateur	Base : localhost, port 15432

Lancer puis examiner, depuis le dossier de compose.yaml

```
docker compose up -d
docker compose ps
docker compose logs api db
docker compose exec db psql -U postgres -c "SELECT 1;"
```

4. Vérifier ce qui tourne

up démarre les services ; -d laisse tourner les programmes en arrière-plan, ce qui rend le terminal disponible. ps affiche leur état. logs lit les messages produits : utile quand un conteneur s'arrête ou qu'une base ne démarre pas. exec exécute une commande dans un conteneur déjà lancé. La dernière ligne demande à PostgreSQL de calculer SELECT 1 : après son démarrage, le résultat attendu est une ligne contenant 1. Le mot de passe d'exemple initialise cette base locale neuve ; il ne sert pas de modèle de secret pour un déploiement.

5. Conserver les fichiers et attendre la disponibilité

Un volume nommé est une zone de stockage gérée par Docker, indépendante du conteneur qui l'utilise. Ici records est relié au dossier où cette version de PostgreSQL écrit ses données. Remplacer db en conservant records permet de retrouver ces données. docker compose down arrête et retire les conteneurs et le réseau, mais garde ce volume ; ajouter -v supprimerait ses données. Un volume n'est pas une sauvegarde : il ne fournit pas à lui seul une copie récupérable après une perte.

Pour approfondir : [12] [14] [13] [18] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Relier deux services et conserver les données

Si cela ne fonctionne pas

- Un conteneur démarré peut encore être indisponible. Un healthcheck contrôle sa disponibilité. `depends_on` simple impose un ordre de démarrage ; `service_healthy` attend un contrôle initial réussi.
- Même après ce contrôle, une panne reste possible : le programme qui utilise la base doit traiter les erreurs de connexion.
- En cas d'échec : regarder `ps`, puis `logs` ; vérifier le nom `db` et le port interne 5432, puis la publication si l'appel vient de l'ordinateur.

L'idée à expliquer en une phrase

Le réseau relie les programmes ; les ports publiés donnent un accès depuis l'ordinateur ; le volume garde les données après le remplacement d'un conteneur.

À TOI DE JOUER · DOC-02

On retire uniquement la ligne `ports` de `db`. Quel accès disparaît ? Que devient `records` après `docker compose down` ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [12] [14] [13] [18] · Références détaillées à la fin.

1 - Vérifier une petite fonction avec un test unitaire

Situation : une réservation doit contenir entre 1 et 5 places, avec un nombre entier. Nous écrivons les résultats attendus avant de tester la fonction qui applique cette règle.

1. Comprendre les mots avant le code

Un test automatisé exécute du code et vérifie un résultat connu. Une assertion est cette comparaison : elle signale une erreur si le résultat réel diffère de l'attendu. Le test runner, ou lanceur de tests, trouve les tests, les exécute et affiche lesquels réussissent ou échouent. Un test unitaire vise une petite unité de logique, ici une fonction, sans serveur ni base à lancer.

La règle donne les attendus

Entrée	Attendu et raison
0	false : inférieur à 1
1	true : première valeur admise
3	true : valeur intérieure
5	true : dernière valeur admise
6	false : supérieur à 5
"3", 1.5 ou NaN	false : ce n'est pas un entier admis

2. Écrire la fonction

Pour pratiquer, il suffit de Node 22 ou plus récent : les outils utilisés sont fournis avec Node. Dans un dossier d'essai, créer `quantite.mjs`. Le premier contrôle exige un entier ; les deux autres imposent les bornes incluses. Le signe `&&` signifie que les trois conditions doivent être vraies. `return` renvoie `true` ou `false` au code qui appelle la fonction ; `export` permet au fichier de test de l'importer.

Fichier `quantite.mjs` complet

```
export function acceptQuantity(value) {
  return Number.isInteger(value)
    && value >= 1
    && value <= 5;
}
```

Pour approfondir : [32] [30] [27] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Vérifier une petite fonction avec un test unitaire

3. Préparer, agir, vérifier

La méthode AAA signifie Arrange, Act, Assert : préparer les données ; appeler le comportement ; comparer le résultat. Nous préparons 0, appelons `acceptQuantity`, puis attendons `false`. Cet attendu vient de la règle annoncée, pas d'une copie du résultat produit par la fonction. Le nom du test explique le comportement examiné ; il aide à comprendre un échec.

Fichier `quantite.test.mjs` complet, dans le même dossier

```
import test from "node:test";
import assert from "node:assert/strict";
import { acceptQuantity } from "../quantite.mjs";
test("refuse une quantité égale à zéro", () => {
  const value = 0;
  const actual = acceptQuantity(value);
  assert.strictEqual(actual, false);
});
```

Terminal : demander à Node de lancer ce fichier

```
node --test quantite.test.mjs
```

4. Lire le résultat et chercher des limites

Résultat attendu : un test réussi, zéro échec. Pour observer un échec, remplacer temporairement `false` dans l'assertion par `true` : le lanceur indique la valeur obtenue et la valeur attendue. Remettre ensuite `false`. Ce test ne vérifie que zéro. Ajouter des tests pour 1, 5 et 6 couvre les deux bornes et leur voisin extérieur ; ajouter une chaîne et un nombre décimal contrôle d'autres sortes d'entrées.

Pour approfondir : [32] [30] [27] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Vérifier une petite fonction avec un test unitaire

5. Choisir ce que l'on compare et ce que l'on remplace

- `strictEqual` compare ici une valeur booléenne. `deepStrictEqual` compare les éléments et propriétés d'un tableau ou d'un objet ; deux objets séparément créés ne sont pas la même référence.
- Les données préparées pour un test sont souvent appelées fixtures. Elles doivent être connues, reproductibles et indépendantes des autres tests.
- Un double de test remplace une dépendance, par exemple un accès aux données, pour contrôler ce qu'elle renvoie. Le test vérifie alors la logique autour de ce double, pas la vraie base.
- Une couverture de code indique ce qui a été parcouru. Elle ne prouve pas que les attendus vérifient correctement toutes les règles.

L'idée à expliquer en une phrase

Un test confronte un résultat réel à un attendu choisi depuis la règle, et les cas limites doivent être testés explicitement.

À TOI DE JOUER · TES-01

La règle devient « entiers de 2 à 8 inclus ». Quelles quatre valeurs testent les bornes et leurs voisins extérieurs ? Quels attendus faut-il écrire ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [32] [30] [27] · Références détaillées à la fin.

2 · Attendre une réponse HTTP et fermer ce qui a été ouvert

Situation : un client demande une tâche à un serveur. Le test vérifie leur échange réel, puis ferme le serveur pour terminer proprement.

1. Passer de l'unité à l'échange

Un test d'intégration vérifie la coopération entre plusieurs éléments. Ici, un vrai client HTTP appelle un vrai serveur local : HTTP est le protocole de leur demande et de leur réponse. Le statut est un code indiquant le résultat ; le corps contient les données. Un test de bout en bout, souvent abrégé E2E pour end-to-end, irait plus loin en vérifiant un parcours utilisateur complet, par exemple depuis une interface.

2. Définir les données et le temps d'attente

Notre serveur connaît exactement une tâche : id 1, titre Lire. GET /tasks/1 donne 200 avec cette tâche ; toute autre demande donne 404 avec error: "Absente". Une opération asynchrone ne fournit pas immédiatement son résultat. Une Promise représente ce résultat futur ; await permet de l'attendre dans une fonction async. Sans cette attente, une comparaison peut arriver trop tôt ou une erreur être observée après la fin du test.

http.test.mjs · Partie 1 : serveur et données, avec Node 22+

```
import test from "node:test";
import assert from "node:assert/strict";
import { createServer } from "node:http";
import { once } from "node:events";
function handleTask(req, res) {
  res.setHeader("Content-Type", "application/json");
  if (req.method === "GET" && req.url === "/tasks/1") {
    res.writeHead(200);
    res.end(JSON.stringify({ id: 1, title: "Lire" }));
    return;
  }
  res.writeHead(404);
  res.end(JSON.stringify({ error: "Absente" }));
}
```

Pour approfondir : [32] [30] [31] [11] [46] [24] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Attendre une réponse HTTP et fermer ce qui a été ouvert

3. Ouvrir, attendre, comparer

Ajouter la partie 2 dans le même fichier, sous la partie 1. `createServer` crée le serveur ; `listen(0)` demande un port libre sur notre ordinateur ; `once` attend que le serveur signale `listening`, c'est-à-dire qu'il écoute. `fetch` envoie la requête. `response.status` fournit le code ; `response.json` attend puis lit le corps JSON. Il faut vérifier les deux : 200 peut accompagner des données incorrectes.

http.test.mjs · Partie 2 : ajouter après la partie 1

```
test("GET /tasks/1 renvoie la tâche connue", async t => {
  const server = createServer(handleTask);
  t.after(() => new Promise(resolve => server.close(resolve)));
  server.listen(0, "127.0.0.1");
  await once(server, "listening");
  const url = "http://127.0.0.1:" + server.address().port;
  const response = await fetch(url + "/tasks/1");
  assert.strictEqual(response.status, 200);
  assert.deepStrictEqual(await response.json(), {
    id: 1, title: "Lire"
  });
});
```

Terminal : les deux parties forment un seul fichier complet

```
node --test http.test.mjs
```

4. Nettoyer même après un échec

`t.after` enregistre une action qui fermera le serveur après ce test, même si une assertion échoue. Ce nettoyage évite de laisser un programme écouter un port ou d'empêcher le processus de terminer. Pour des tests avec une vraie base, utiliser une base dédiée, des données connues et des connexions libérées ; ne pas dépendre d'une base contenant les données réelles des utilisateurs. `fetch` ne considère pas automatiquement 404 comme une Promise rejetée : le statut doit être vérifié.

Pour approfondir : [32] [30] [31] [11] [46] [24] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Attendre une réponse HTTP et fermer ce qui a été ouvert

5. Situer Vitest et la méthode TDD

Vitest est un autre lanceur de tests JavaScript. Il utilise souvent expect : l'assertion `expect(actual).toBe(false)` correspond à la comparaison de valeur précédente ; `toStrictEqual` compare une structure avec des contrôles stricts. Ce sont d'autres écritures de la même intention. TDD signifie test-driven development, développement guidé par les tests : écrire un test qui échoue pour le comportement manquant, écrire le code qui le fait réussir, puis améliorer le code en gardant les tests réussis. On appelle ces étapes Red, Green, Refactor.

L'idée à expliquer en une phrase

Pour tester un échange asynchrone, j'attends son résultat, je contrôle le statut et les données, puis je ferme les ressources ouvertes.

À TOI DE JOUER · TES-02

Adapter uniquement la demande et les assertions pour `GET /tasks/99` : quels sont le statut et le corps attendus ? Le test vérifie-t-il une vraie base ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [32] [30] [31] [11] [46] [24] · Références détaillées à la fin.

1 · Le client demande, le serveur répond : HTTP

Situation : une interface veut afficher la tâche Lire. Le backend est le programme côté serveur qui traite sa demande ; nous commençons par un échange avec une seule tâche connue.

1. Identifier les deux interlocuteurs

Le client est le programme qui demande quelque chose : navigateur, interface ou outil de test. Le serveur est le programme qui écoute et répond. HTTP est leur protocole de communication, c'est-à-dire les règles de leurs demandes et réponses. Une API, interface de programmation, offre des opérations utilisables par d'autres programmes. Une interface graphique peut utiliser une API, mais une API peut aussi être appelée sans écran.

Les parties d'une demande et d'une réponse

Partie	Dans notre exemple
Méthode de la requête	GET : demander une lecture.
Chemin de la requête	/tasks/1 : demander la tâche identifiée par 1.
En-têtes	Informations sur le message, comme son format.
Corps	Données éventuellement envoyées ou reçues.
Statut de la réponse	200 : demande traitée avec succès ; 404 : absente.

2. Donner un contrat précis

Une route associe une méthode et un chemin à un traitement. Notre contrat contient GET /tasks/1 : réponse 200 et {"id":1,"title":"Lire"}. Les autres demandes donnent ici 404 et {"error":"Absente"}. Le corps est du JSON, un format textuel de données : les noms de propriétés sont entre guillemets doubles. Une réponse n'est pas seulement ce corps ; son statut dit aussi si le traitement a réussi.

3. Lire le programme avant de le lancer

Ce programme autonome utilise uniquement Node 22 ou plus récent, qui exécute JavaScript. Créer lecture.mjs avec le bloc suivant. createServer reçoit une fonction appelée pour chaque requête : req contient la demande ; res sert à fabriquer la réponse. Notre donnée est écrite directement dans ce programme, sans base ni installation supplémentaire. Le serveur écoute le port 3000 sur 127.0.0.1, donc seulement depuis notre ordinateur.

Pour approfondir : [05] [31] [70] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Le client demande, le serveur répond : HTTP

Fichier lecture.mjs complet

```
import { createServer } from "node:http";
createServer((req, res) => {
  res.setHeader("Content-Type", "application/json");
  if (req.method === "GET" && req.url === "/tasks/1") {
    res.writeHead(200);
    res.end(JSON.stringify({ id: 1, title: "Lire" }));
    return;
  }
  res.writeHead(404);
  res.end(JSON.stringify({ error: "Absente" }));
}).listen(3000, "127.0.0.1");
```

4. Comprendre les lignes qui décident

La condition examine la méthode et le chemin : les deux doivent correspondre. `writeHead` choisit le statut de la réponse. `JSON.stringify` transforme un objet JavaScript en texte JSON ; `res.end` envoie ce texte et termine la réponse. `return` arrête ce traitement après le succès : il évite de tenter ensuite une deuxième réponse 404. `Content-Type: application/json` annonce au client comment lire les données reçues.

Lancer dans un terminal ; demander dans un second terminal

```
node lecture.mjs
# Depuis le second terminal :
curl -i http://127.0.0.1:3000/tasks/1
curl -i http://127.0.0.1:3000/tasks/99
```

5. Observer avant d'ajouter des outils

`curl` est un client HTTP en ligne de commande ; `-i` affiche le statut et les en-têtes avec le corps. Le premier appel affiche 200 et la tâche Lire ; le second affiche 404 et l'erreur Absente. Le navigateur peut aussi ouvrir ces adresses, mais le corps seul n'explique pas tout le résultat. Arrêter le serveur avec `Ctrl+C` avant d'utiliser le même port dans la leçon suivante. Si 3000 est occupé, le serveur ne démarre pas ; ce n'est pas une erreur de données.

Pour approfondir : [05] [31] [70] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Le client demande, le serveur répond : HTTP

L'idée à expliquer en une phrase

Une route reçoit une méthode et un chemin, puis renvoie un statut et des données selon un contrat connu.

À TOI DE JOUER · BAC-01

GET /tasks/99 renvoie {"error":"Absente"}. Quel statut doit accompagner ce corps ? Que provoquerait l'absence de return après la réponse de succès dans ce code ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [05] [31] [70] · Références détaillées à la fin.

2 · Répartir le traitement : middleware, contrôleur, service

Situation : nous gardons la même lecture, mais organisons le traitement pour savoir où modifier une règle, une réponse HTTP ou un accès aux données.

1. Donner un rôle à chaque étape

Un middleware est une fonction intermédiaire qui reçoit la requête et la réponse. Il peut répondre, préparer des informations, ou transmettre à l'étape suivante avec `next()`. Un contrôleur adapte l'échange HTTP : il lit les paramètres et choisit la réponse. Un service porte ou orchestre les règles du traitement. Un repository regroupe la lecture et l'écriture des données. Ces rôles sont une manière d'organiser le code ; ils ne nécessitent pas trois serveurs.

2. Préparer notre version Express

Express aide Node à organiser routes et middleware. Pour pratiquer, créer un dossier d'essai : `npm init -y`, puis `npm install express@5`. La version choisie correspond au comportement des erreurs expliqué plus bas. Réunir les trois parties dans `api.mjs`, dans l'ordre. Les données restent un tableau : cet exercice ne lit aucune base.

api.mjs · Partie 1 : préparer les étapes et la donnée

```
import express from "express";
const app = express();
app.use(express.json());
app.use((req, res, next) => {
  console.log(req.method, req.path);
  next();
});
const tasks = [{ id: 1, title: "Lire" }];
const repo = { find: id => tasks.find(task => task.id === id) };
const service = id => repo.find(id);
```

3. Lire l'ordre de passage

`app.use` ajoute ici des middleware avant les routes. `express.json` decode le corps JSON dans `req.body` ; un JSON mal écrit provoque une erreur 400. Le middleware suivant écrit méthode et chemin dans le terminal, puis `next` laisse continuer. Sans réponse et sans `next`, la requête reste en attente. Le service effectue seulement une lecture ici ; il pourrait aussi appliquer des règles.

Pour approfondir : [21] [20] [25] [26] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Répartir le traitement : middleware, contrôleur, service

api.mjs · Partie 2 : adapter le paramètre et répondre

```
app.get("/tasks/:id", (req, res) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id < 1) {
    return res.status(400).json({ error: "Id invalide" });
  }
  const task = service(id);
  if (!task) return res.status(404).json({ error: "Absente" });
  res.status(200).json(task);
});
```

4. Séparer le transport de la recherche

Dans `/tasks/:id`, `id` est un segment variable. `req.params.id` contient un texte, par exemple `"1"` ; `Number` le convertit. Le contrôle refuse les nombres non entiers ou inférieurs à 1. Le contrôleur appelle le service, qui appelle `repo.find`. `res.status` choisit le code ; `json` envoie le corps. `/tasks/1` donne 200 et Lire ; `/tasks/99` donne 404 ; `/tasks/abc` donne 400.

api.mjs · Partie 3 : erreurs en dernier, puis démarrage

```
app.use((err, req, res, next) => {
  if (res.headersSent) return next(err);
  let status = err.status || err.statusCode;
  if (!Number.isInteger(status) || status < 400 || status > 599) {
    status = 500;
  }
  res.status(status).json({ error: "Requête refusée ou incident" });
});
app.listen(3000, "127.0.0.1");
```

5. Ne pas transformer une mauvaise entrée en panne serveur

Le middleware d'erreur a quatre paramètres et suit les routes. Il conserve un statut connu, dont le 400 du décodeur JSON ; sinon, il répond 500. Si la réponse a commencé, `headersSent` est vrai : on transmet à Express pour éviter une deuxième réponse. Express 5 transmet aussi une exception ou le rejet d'une Promise retournée par un traitement `async`.

Pour approfondir : [21] [20] [25] [26] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Répartir le traitement : middleware, contrôleur, service

Après `node api.mjs` : vérifier aussi le JSON mal formé

```
curl -i http://127.0.0.1:3000/tasks/1
curl -i -X POST http://127.0.0.1:3000/tasks/1 \
  -H "Content-Type: application/json" -d '{}'
```

L'idée à expliquer en une phrase

Le contrôleur parle HTTP, le service traite la demande, le repository accède aux données ; les middleware préparent ou interrompent ce parcours.

À TOI DE JOUER · BAC-02

Le second appel curl envoie un corps `{` incomplet. Quel statut est attendu ? Si le stockage passe d'un tableau à une base, quel rôle doit porter le nouvel accès ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [21] [20] [25] [26] · Références détaillées à la fin.

3 - Valider une donnée reçue avant de l'utiliser

Situation : une réservation contient seats, le nombre de places. Nous exigeons un entier de 1 à 4 ; une valeur envoyée par le client n'est jamais correcte simplement parce qu'elle est arrivée.

1. Distinguer une donnée lisible et une donnée admise

Décoder le JSON vérifie que le texte respecte ce format. Valider vérifie ensuite sa forme et ses valeurs selon les règles de l'opération. {"seats": "2"} est un JSON lisible, mais seats est un texte ; notre règle exige un entier de 1 à 4. Le client peut contourner son formulaire : le serveur refait les vérifications.

Le contrat de réservation, avant le code

Corps envoyé	Admis ?
{"seats": 2}	Oui : entier entre 1 et 4.
{"seats": "2"}	Non : texte au lieu d'un nombre.
{"seats": 5}	Non : quantité supérieure à 4.
{"seats": 1.5}	Non : nombre non entier.
{}	Non : seats manque.
{	Le texte ne peut même pas être décodé en JSON.

2. Décrire la règle avec Zod

Zod est une bibliothèque qui vérifie des valeurs pendant l'exécution à partir d'une description attendue, appelée schéma. Ici object exige un objet ; number un nombre ; int un entier ; min et max les bornes. Ces contrôles ne transforment pas automatiquement "2" en nombre : notre contrat refuse ce texte.

validation.mjs · Exemple autonome après npm install zod@4

```
import { z } from "zod";
const booking = z.object({
  seats: z.number().int().min(1).max(4)
});
const bodies = [{ seats: 2 }, { seats: "2" }, { seats: 5 }];
for (const body of bodies) {
  const result = booking.safeParse(body);
  console.log(result.success);
}
```

Pour approfondir : [80] [72] [70] · Références détaillées à la fin.

SUITE DE LA LEÇON

3 · Valider une donnée reçue avant de l'utiliser

3. Lire les résultats de validation

Avec Node, lancer `node validation.mjs` : les résultats attendus sont `true`, `false`, `false`, dans cet ordre. `safeParse` renvoie un objet résultat. Si `success` vaut `true`, `data` contient les données validées à utiliser ensuite. Sinon, `error` décrit les problèmes et le traitement ne doit pas enregistrer la réservation. Ce contrôle n'a encore créé aucune réservation : notre fichier démontre seulement la validation.

Fragment à lire : `body` représente les données déjà décodées

```
const result = booking.safeParse(body);
if (!result.success) {
  return res.status(422).json({ error: "Places invalides" });
}
const seats = result.data.seats;
// Le service peut maintenant utiliser seats, entier de 1 à 4.
```

4. Placer la vérification dans le parcours

Ce bloc est un fragment de contrôleur : `booking` est le schéma précédent, `body` la donnée reçue, `res` la réponse Express. `return` arrête le traitement en cas d'échec. Après réussite, utiliser `result.data`. Le service vérifie ensuite d'autres règles, par exemple les places disponibles ; une donnée correctement formée ne garantit pas cette disponibilité.

Choisir une réponse selon le résultat

Situation	Statut de notre contrat
Texte JSON incompréhensible	400 : la demande ne peut pas être lue.
JSON lisible, <code>seats</code> invalide	422 : le contenu ne respecte pas la règle.
Donnée valide et réservation effectivement créée	201 : création réalisée.

5. Situer TypeScript sans le confondre avec ce contrôle

TypeScript examine statiquement le code avant l'exécution ; ses annotations sont ensuite effacées. Annoncer un type pour `body`, ou l'affirmer avec `as`, ne contrôle pas un JSON reçu. La validation vérifie effectivement cette valeur. Les types aident à écrire le programme ; les tests vérifient des comportements ; la validation contrôle les données reçues : ces vérifications se complètent.

Pour approfondir : [80] [72] [70] · Références détaillées à la fin.

SUITE DE LA LEÇON

3 · Valider une donnée reçue avant de l'utiliser

L'idée à expliquer en une phrase

Décoder rend le texte lisible ; valider vérifie sa forme et ses valeurs ; seule la donnée validée peut poursuivre le traitement.

À TOI DE JOUER · BAC-03

Le client envoie `{"seats": "2"}`. Le JSON se décode bien et le type annoncé dans notre code était `{seats: number}`. Peut-on créer la réservation ? Quel résultat produit `safeParse` ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [80] [72] [70] · Références détaillées à la fin.

4 · Identifier une personne puis vérifier son droit d'agir

Situation : le document 42 appartient à l'utilisateur 9. L'utilisateur 7 est bien connecté ; cela ne lui donne pas le droit de modifier ce document.

1. Deux questions différentes

L'authentification établit une identité : « qui demande ? ». L'autorisation décide d'une permission : « cette personne peut-elle faire cette action sur cette ressource ? ». Cacher un bouton ne protège pas une route appelable directement. Ici, seul le propriétaire enregistré peut modifier le document.

2. Vérifier un mot de passe sans le conserver en clair

Lors d'une connexion par mot de passe, le serveur vérifie la preuve fournie. Il conserve un hachage adapté, par exemple obtenu avec Argon2id, jamais le mot de passe lisible. Ce hachage permet la vérification sans être un chiffrement que l'on déchiffre ensuite. Un sel est une valeur aléatoire propre à chaque mot de passe : le même mot de passe produit des hachages différents selon ce sel. La bibliothèque génère et conserve ce sel avec le hachage ; il n'est pas secret.

3. Comprendre ce que transporte un JWT

Après authentification, le serveur peut délivrer un JWT, JSON Web Token : un jeton contenant identifiant, émetteur et expiration. La signature permet de vérifier son origine et l'absence de modification. Le contenu reste lisible, donc sans secrets. decode lit sans prouver l'authenticité ; verify contrôle signature et conditions configurées.

Fragment à lire : vérifier un jeton selon notre contrat

```
const identity = jwt.verify(token, publicKey, {
  algorithms: ["RS256"],
  issuer: "https://identite.exemple",
  audience: "bibliotheque-api"
});
if (typeof identity.exp !== "number") {
  throw new Error("Expiration requise");
}
```

Pour approfondir : [02] [03] [22] [06] [69] [68] · Références détaillées à la fin.

SUITE DE LA LEÇON

4 · Identifier une personne puis vérifier son droit d'agir

4. Donner le contexte et les résultats de verify

Ce fragment suppose jwt fourni par jsonwebtoken, token reçu et publicKey issu de la configuration fiable du serveur. algorithms impose ici la méthode de signature RS256 ; issuer impose l'émetteur ; audience impose l'API destinataire. verify refuse une signature incorrecte ou ces valeurs incompatibles. Il contrôle exp, l'expiration, lorsqu'elle est présente ; notre contrat exige aussi sa présence. Vérifier également la forme de l'identifiant attendu. Ne jamais attribuer des droits depuis un jeton refusé.

Appliquer les droits avec les données du serveur

Données connues	Décision
Identité absente ou jeton invalide	401 : identité à établir.
Identité 7 valide ; document 42, propriétaire 9	403 : modification interdite.
Identité 9 valide ; document 42, propriétaire 9	Modification permise par cette règle.

5. Lire le propriétaire fiable et annoncer une identité manquante

Le propriétaire vient des données du serveur : ownerId:7 envoyé dans le corps ne remplace pas le propriétaire 9 et ne crée aucun droit. Une réponse 401 inclut WWW-Authenticate, l'en-tête annonçant le mécanisme attendu. Avec un jeton Bearer, on peut annoncer Bearer realm="bibliotheque". Une identité valide sans permission conduit ici à 403.

Fragment de réponse Express pour une identité manquante

```
res.set("WWW-Authenticate", 'Bearer realm="bibliotheque"');  
res.status(401).json({ error: "Identité requise" });
```

Pour approfondir : [02] [03] [22] [06] [69] [68] · Références détaillées à la fin.

SUITE DE LA LEÇON

4 · Identifier une personne puis vérifier son droit d'agir

6. Si le navigateur transporte l'identité dans un cookie

- Un cookie est une donnée conservée et éventuellement envoyée automatiquement par le navigateur. HttpOnly empêche sa lecture JavaScript ; Secure réserve son envoi à HTTPS.
- CSRF signifie falsification de requête intersite : une autre page tente de déclencher une action via un cookie automatiquement envoyé. Protéger les actions sensibles ; SameSite limite certains envois et complète ces protections.
- Ne pas enregistrer mots de passe, jetons ou clés secrètes dans les logs, les messages de diagnostic.

L'idée à expliquer en une phrase

L'identité vérifiée dit qui demande ; les règles et données du serveur décident si cette personne a le droit d'agir.

À TOI DE JOUER · BAC-04

L'utilisateur 7 a un JWT valide et envoie ownerId:7 pour modifier le document 42, dont le propriétaire enregistré est 9. Quelle réponse et quelle modification faut-il effectuer ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [02] [03] [22] [06] [69] [68] · Références détaillées à la fin.

5 · Lire des données : SQL, ORM et changements cohérents

Situation : nos tâches sont désormais conservées dans une base. Nous comparons deux manières de les lire, puis expliquons comment éviter des données incohérentes.

1. Décrire les données avant les outils

Une base relationnelle conserve des lignes dans des tables avec des colonnes et des règles. task contient id, title et done, qui indique si la tâche est terminée. id est la clé primaire : identité sans doublon ni absence. Le tableau donne toutes nos données. Les blocs supposent une connexion PostgreSQL : ce sont des fragments expliqués, pas une application prête à lancer.

Toutes les lignes de task

id	title	done
1	Lire	false
2	Marcher	true

2. Donner une instruction SQL et une valeur séparée

SQL exprime une lecture ou une modification. SELECT choisit les colonnes, FROM la table, WHERE les lignes. Le programme envoie la valeur séparément de l'instruction : \$1 désigne le premier paramètre, qui ne devient pas du code SQL. Valider aussi l'identifiant comme entier : la séparation protège la requête ; la validation protège les règles du traitement.

Fragment avec db, un client PostgreSQL déjà connecté

```
const id = 1;
const { rows } = await db.query(
  "SELECT id, title FROM task WHERE id = $1", [id]
);
// rows : [{ id: 1, title: "Lire" }]
```

3. Lire la même base avec un ORM

ORM signifie Object-Relational Mapping : un outil associe données relationnelles et objets, puis propose des méthodes d'accès. Prisma connaît ici un modèle Task associé à task, avec les colonnes données. Son client généré, prisma, expose ces méthodes. Ces fragments suivent Prisma 6.19.0 ; ils expliquent la lecture, sans configuration d'une nouvelle application.

Pour approfondir : [29] [48] [67] [66] [65] · Références détaillées à la fin.

SUITE DE LA LEÇON

5 · Lire des données : SQL, ORM et changements cohérents

Fragment Prisma : ne garder que les tâches non terminées

```
const todo = await prisma.task.findMany({
  where: { done: false },
  orderBy: { id: "asc" },
  select: { title: true }
});
// todo : [{ title: "Lire" }]
```

4. Comprendre les options, puis les contraintes

`findMany` renvoie un tableau, même vide. `where` filtre `done:false` ; `orderBy` ordonne les identifiants ; `select` garde seulement `title`. `findUnique` cherche un identifiant unique : ligne ou null. `skip` et `take` sautent puis limitent les lignes ordonnées ; `include` ajoute une relation du modèle. Malgré l'ORM, la base impose ses contraintes : clé étrangère pour une référence existante, `UNIQUE` contre les doublons, `NOT NULL` pour exiger une valeur.

5. Faire évoluer la structure et grouper les changements

Une migration est une modification enregistrée de la structure de la base, par exemple l'ajout d'une colonne. `migrate dev` aide à créer et appliquer ces changements en développement ; `migrate deploy` applique les migrations déjà écrites au déploiement. `generate` fabrique le client Prisma à partir du modèle ; il ne crée pas les tables. Une transaction regroupe des écritures : elles sont toutes validées, ou toutes annulées si le groupe échoue.

Fragment Prisma : groupe dont la seconde opération échoue

```
await prisma.$transaction([
  prisma.task.update({
    where: { id: 1 }, data: { title: "Renommée" }
  }),
  prisma.task.delete({ where: { id: 99 } })
]);
```

6. Expliquer ce qui reste après l'échec

La tâche 99 est absente : cette suppression Prisma échoue. La transaction annule le renommage de 1 : son titre reste Lire. En SQL ordinaire, supprimer une ligne absente ne provoquerait pas à elle seule cette erreur. Une transaction ne remplace pas les contraintes métier ni l'isolation adaptée aux modifications simultanées.

Pour approfondir : [29] [48] [67] [66] [65] · Références détaillées à la fin.

SUITE DE LA LEÇON

5 · Lire des données : SQL, ORM et changements cohérents

L'idée à expliquer en une phrase

Le paramètre sépare une valeur du SQL ; l'ORM organise cet accès ; contraintes et transactions protègent la cohérence des données.

À TOI DE JOUER · BAC-05

Avec les deux lignes données, que renvoie findMany avec where:{done:false} et select:{title:true} ?
Après la transaction en échec, quel est le titre de la tâche 1 ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [29] [48] [67] [66] [65] · Références détaillées à la fin.

1 · Un composant décrit l'écran avec des props

Objectif : reconnaître JSX, un composant et ses entrées. React aide à construire une interface en partant des données ; commençons par un simple message.

Un composant est une fonction d'interface

Une fonction ordinaire peut calculer un nombre. Un composant React est une fonction qui décrit une partie de l'interface : un message, un champ, une liste. Son nom commence par une majuscule lorsqu'il est utilisé en JSX. JSX est une syntaxe proche de balises HTML dans le JavaScript. `<p>Bonjour</p>` décrit un paragraphe. JSX n'est pas directement compris comme tel par le navigateur : l'environnement React le transforme en JavaScript. Les fragments `App.jsx` supposent un environnement React qui transforme JSX et affiche `<App />` dans une page. Il manque donc l'initialisation de cette page et cet outil de transformation ; le guide officiel cité montre comment les préparer. Aucun serveur de données personnel n'est utilisé.

Texte fixe ou donnée calculée ?

Dans `<p>Bonjour Alice</p>`, tout est fixe. Les accolades JSX permettent d'insérer le résultat d'une expression JavaScript : `<p>Bonjour {nom}</p>` lit la valeur `nom`. Elles ne servent pas ici à créer un objet. Une balise en minuscules, comme `<p>`, décrit un élément du navigateur. Une balise en majuscule, comme `<Bonjour />`, utilise notre composant `Bonjour`. Le signe `/>` ferme une balise sans contenu enfant.

Une prop est une entrée fournie par le parent

Le composant parent emploie son enfant et lui fournit des valeurs. Ces entrées sont appelées props, abréviation de properties, propriétés. `<Bonjour nom="Alice" />` transmet une propriété `nom` contenant le texte `Alice`. La fonction `Bonjour` reçoit un objet nommé ici `props` et lit `props.nom`. Le composant peut être réutilisé avec une autre valeur sans dupliquer son fonctionnement. Il ne doit pas modifier les props reçues.

Fragment `App.jsx` · un parent et son composant enfant

```
function Bonjour(props) {
  return <p>Bonjour {props.nom}</p>;
}
export default function App() {
  return <Bonjour nom="Alice" />;
}
```

Pour approfondir : [35] [34] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Un composant décrit l'écran avec des props

Suivre l'entrée jusqu'à l'affichage

App décrit un Bonjour avec nom égal à Alice. React appelle Bonjour avec les props correspondantes. props.nom lit Alice, les accolades insèrent ce texte, et le paragraphe affiché est Bonjour Alice.

Remplacer le parent par `<Bonjour nom="Bilal" />` produirait Bonjour Bilal. export default rend App disponible comme composant principal pour le fichier qui initialise l'affichage ; ce n'est pas un bouton ou un effet visible.

Ce que signifie rendre un composant

Le rendu, render en anglais, est le calcul de la description de l'écran à partir des données. React compare ensuite le résultat et met à jour l'interface si nécessaire. Rendre ne signifie pas recharger toute la page ni détruire systématiquement tous ses éléments. Pour le moment les props sont fixes ; la prochaine leçon ajoutera une donnée qui change. Un composant ne doit pas lancer une opération réseau ou modifier une donnée extérieure simplement parce qu'il calcule son rendu.

Même enfant · deux usages possibles dans App

```
export default function App() {  
  return (  
    <main>  
      <Bonjour nom="Alice" />  
      <Bonjour nom="Bilal" />  
    </main>  
  );  
}
```

Pour approfondir : [35] [34] · Références détaillées à la fin.

SUITE DE LA LEÇON

1 · Un composant décrit l'écran avec des props

Ce dernier fragment demande exactement quoi ?

Il remplace la fonction App du premier bloc, et conserve la définition de Bonjour placée au-dessus. Les parenthèses permettent de présenter le JSX sur plusieurs lignes ; `<main>` regroupe les deux paragraphes. Le résultat attendu est Bonjour Alice puis Bonjour Bilal. Tous les noms utilisés dans ce fragment sont ainsi définis ; il manque seulement l'environnement React décrit au début de la leçon.

À TOI DE JOUER · REA-01

Composant fourni : `function Message(props) { return <p>{props.texte}</p>; }`. Le parent utilise `<Message texte="Prêt" />`, puis `<Message texte="Terminé" />`. Quels textes apparaissent ? Dans `{props.texte}`, les accolades créent-elles un objet ? Pourquoi Message commence-t-il par une majuscule ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [35] [34] · Références détaillées à la fin.

2 · Un événement met à jour l'état, puis React rend à nouveau

Objectif : suivre un clic et une saisie jusqu'au nouvel affichage. Prérequis : un composant reçoit des props et retourne une description en JSX.

Pourquoi une variable locale ne suffit-elle pas ?

Une interface doit conserver certaines valeurs entre ses rendus : compteur, texte saisi ou sélection. Cette mémoire s'appelle l'état, state en anglais. `useState` est une fonction fournie par React pour obtenir cette mémoire et demander sa mise à jour ; on parle de hook. Elle est appelée en haut du composant, pas seulement dans une condition. `import` récupère ici cette fonction depuis la bibliothèque `react`.

Lire les deux éléments de `useState`

`useState(0)` renvoie deux éléments : la valeur actuelle et une fonction pour demander la suivante. `const [nombre, setNombre]` donne un nom à chacun en extrayant les deux positions du tableau retourné. 0 est la valeur initiale. Appeler `setNombre` ne change pas rétroactivement la valeur nombre déjà lue dans le rendu courant : React prépare un nouveau rendu avec le nouvel état.

Ce qui se passe au clic

Un événement est une action signalée par le navigateur, comme un clic ou une saisie. `onClick` reçoit ici une fonction : `ajouter`, sans parenthèses. React l'appellera au clic. Écrire `ajouter()` dans cette prop l'appellerait pendant le rendu, au mauvais moment. `precedent => precedent + 1` est une fonction fléchée : elle reçoit l'état précédent et calcule le suivant. Cette forme s'appelle un `updater` et convient quand le calcul dépend de l'ancienne valeur.

Fragment `App.jsx` · un compteur et un événement clic

```
import { useState } from 'react';
export default function App() {
  const [nombre, setNombre] = useState(0);
  function ajouter() {
    setNombre(precedent => precedent + 1);
  }
  return (
    <button onClick={ajouter}>Compteur : {nombre}</button>
  );
}
```

Pour approfondir : [40] [38] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Un événement met à jour l'état, puis React rend à nouveau

Chronologie du compteur

Moment	État et texte affiché
Premier rendu	nombre = 0 ; Compteur : 0.
Premier clic	L'updater calcule 0 + 1 ; nouveau rendu : 1.
Deuxième clic	Il calcule 1 + 1 ; nouveau rendu : 2.

Un champ contrôlé conserve une seule source

Pour un champ texte, value fixe ce qui doit apparaître. onChange est appelé quand la saisie change ; l'événement fourni donne la nouvelle valeur dans event.target.value. Un champ contrôlé a sa valeur dans l'état React : le navigateur signale une nouvelle saisie, le composant enregistre cette valeur, puis un nouveau rendu la remet dans le champ. Cela permet à l'aperçu de rester cohérent avec la saisie.

Autre fragment App.jsx indépendant · saisir et afficher

```
import { useState } from 'react';
export default function App() {
  const [nom, setNom] = useState('');
  function changer(event) {
    setNom(event.target.value);
  }
  return (
    <main>
      <input aria-label="Prénom" value={nom} onChange={changer} />
      <p>Bonjour {nom}</p>
    </main>
  );
}
```

Pour approfondir : [40] [38] · Références détaillées à la fin.

SUITE DE LA LEÇON

2 · Un événement met à jour l'état, puis React rend à nouveau

Relire le mouvement de la donnée

Au début, nom vaut le texte vide : le champ est vide et le paragraphe dit Bonjour. Saisir Alice fournit cette nouvelle valeur à changer ; `setNom` demande son enregistrement ; le prochain rendu affiche Alice dans le champ et Bonjour Alice dans le paragraphe. `aria-label` nomme le champ pour l'accessibilité. Sans `onChange` qui met à jour l'état, `value` continuerait à imposer l'ancienne valeur. Ces deux exemples sont indépendants : chacun remplace App, dans l'environnement défini en leçon 1.

À TOI DE JOUER · REA-02

Le compteur fourni commence à 0. Son gestionnaire appelle deux fois `setNombre(n => n + 1)` pendant un même clic. Quel résultat faut-il attendre ? Pour le champ fourni, si `event.target.value` vaut "Bilal", que doit afficher le prochain rendu ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [40] [38] · Références détaillées à la fin.

3 · Choisir le propriétaire de l'état et identifier les éléments

Objectif : savoir où conserver une donnée utilisée à plusieurs endroits, puis mettre à jour une liste sans perdre l'identité de ses éléments. Prérequis : props, état et gestionnaire d'événement.

Deux composants doivent lire la même valeur

Un champ et un aperçu affichent tous deux le même prénom. Si chacun conservait son propre état, ils pourraient se contredire. On place donc la valeur dans leur parent commun, le composant qui les utilise tous les deux. Cette organisation s'appelle remonter l'état, *lifting state up*. Le parent transmet la valeur à l'enfant, ainsi qu'une fonction qui permettra à l'enfant de signaler un changement.

Fragment · enfant Champ, à conserver au-dessus de App

```
function Champ(props) {  
  function changer(event) {  
    props.onNom(event.target.value);  
  }  
  return (  
    <input aria-label="Prénom"  
      value={props.nom} onChange={changer} />  
  );  
}
```

Un enfant peut recevoir une fonction

`onNom` est une prop comme `nom`, mais sa valeur est une fonction. Quand la saisie change, `Champ` lui fournit le nouveau texte. L'enfant ne possède pas lui-même la mémoire `nom` ; il avertit celui qui la possède. Le nom `onNom` est une convention choisie pour cet exemple. `onChange` est en revanche la prop d'événement reconnue sur l'élément `input`. Ne pas confondre ces deux niveaux.

Fragment App.jsx · avec Champ du bloc précédent

```
import { useState } from 'react';  
export default function App() {  
  const [nom, setNom] = useState('');  
  return (  
    <main>  
      <Champ nom={nom} onNom={setNom} />  
      <p>Aperçu : {nom}</p>  
    </main>  
  );  
}
```

Pour approfondir : [36] [35] [40] [37] · Références détaillées à la fin.

SUITE DE LA LEÇON

3 · Choisir le propriétaire de l'état et identifier les éléments

Suivre Alice depuis le champ

La saisie Alice provoque changer dans Champ. Il appelle props.onNom("Alice"), qui est la fonction setNom du parent. L'état du parent devient Alice. React rend App à nouveau, transmet nom="Alice" à Champ et produit Aperçu : Alice. La source de la valeur est unique. Il manque seulement l'environnement React de leçon 1 ; la définition de Champ et les imports nécessaires sont fournis ici.

Une liste demande une identité stable

map crée un nouveau tableau en calculant un résultat pour chaque élément. En JSX, on peut s'en servir pour obtenir une liste de balises. React demande alors key pour reconnaître chaque élément entre deux rendus. Un identifiant stable convient mieux que la position dans le tableau : après une suppression ou un changement d'ordre, l'élément reste le même. key aide React ; cette prop n'est pas automatiquement reçue par l'enfant comme une donnée métier.

Fragment · code à placer dans le corps et le return de App

```
const activites = [
  { id: 10, titre: 'Lire' },
  { id: 20, titre: 'Marcher' }
];
// JSX à retourner :
<ul>
  {activites.map(item => (
    <li key={item.id}>{item.titre}</li>
  ))}
</ul>
```

Préciser les deux parties de ce fragment

La déclaration activites va avant le return du composant ; le bloc est son JSX retourné. Ce n'est pas un programme à coller d'un seul tenant au milieu de return. item est le nom donné à l'élément actuellement parcouru par map. Le résultat est deux lignes : Lire et Marcher. Leurs clés sont 10 et 20, et non 0 et 1.

Pour approfondir : [36] [35] [40] [37] · Références détaillées à la fin.

SUITE DE LA LEÇON

3 · Choisir le propriétaire de l'état et identifier les éléments

Modifier une liste d'état sans mutation

Une mutation modifie directement un objet existant, par exemple supprimer un élément avec splice. Pour un tableau conservé dans l'état, produire plutôt une nouvelle valeur. filter crée un nouveau tableau contenant les éléments qui satisfont une condition. Le symbole !== signifie « différent de ». Supprimer l'id 10 revient à garder les éléments dont l'id n'est pas 10. Dans un composant avec const [activites, setActivites] = useState(...), transmettre ce nouveau tableau à setActivites.

Programme JavaScript autonome · observer la nouvelle liste

```
const avant = [
  { id: 10, titre: 'Lire' },
  { id: 20, titre: 'Marcher' }
];
const apres = avant.filter(item => item.id !== 10);
console.log(avant.length, apres.length, apres[0].titre);
// Résultat : 2 1 Marcher.
```

À TOI DE JOUER · REA-03

Données : le parent possède nom = "Alice" et le transmet au champ et à l'aperçu. L'enfant signale onNom("Bilal"). Qui modifie l'état ? Autre liste : ids 10/Lire et 20/Marcher. Après suppression de 10, quelle key doit conserver Marcher et pourquoi ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [36] [35] [40] [37] · Références détaillées à la fin.

4 · Un effet synchronise une ressource extérieure au rendu

Objectif : lire `useEffect`, ses dépendances et sa fonction de nettoyage. Prérequis : distinguer le calcul d'un écran de la modification d'une donnée.

Un cas qui ne se résout pas seulement avec JSX

Le paragraphe Bonjour Alice est calculé depuis une prop ; aucun effet n'est nécessaire pour cela. Nous voulons maintenant changer le titre de l'onglet du navigateur. Ce titre est extérieur au contenu JSX retourné par notre composant. `document` est l'objet fourni par le navigateur pour accéder à la page ; `document.title` lit ou remplace ce titre. `useEffect` permet de synchroniser une ressource de ce genre avec les valeurs du composant.

Lire les deux arguments avant le code

`useEffect` reçoit une fonction décrivant la synchronisation et un tableau de dépendances. Une dépendance est une valeur réactive utilisée par l'effet : une prop, un état ou une valeur calculée dans le composant. Si elle change entre deux rendus, React doit resynchroniser. Dans `[nom]`, les crochets forment le tableau contenant notre prop `nom`. Ce tableau n'est pas la liste des actions à effectuer.

Fragment · composant `Titree`, recevant `nom` comme prop

```
import { useEffect } from 'react';
function Titree(props) {
  const nom = props.nom;
  useEffect(() => {
    const ancien = document.title;
    document.title = 'Lecture de ' + nom;
    return () => {
      document.title = ancien;
    };
  }, [nom]);
  return <p>Bonjour {nom}</p>;
}
```

Ce qui manque pour l'exécuter

Le fragment fournit tout le composant, mais pas son affichage dans l'application. Dans l'environnement React de leçon 1, un parent peut retourner `<Titree nom="Alice" />`. Il faut un navigateur, car `document.title` lui appartient. Au départ, supposons que l'onglet s'appelle CDA. React installe le composant, puis exécute l'effet : il mémorise CDA et remplace le titre par Lecture de Alice.

Pour approfondir : [39] [35] · Références détaillées à la fin.

SUITE DE LA LEÇON

4 · Un effet synchronise une ressource extérieure au rendu

Le return de l'effet ne retourne pas un écran

`return <p>...` appartient au composant : il décrit son écran. `return () => { ... }` appartient à l'effet : il fournit à React une fonction de nettoyage. React la conserve pour annuler ou terminer la synchronisation précédente. Il la lance avant une nouvelle synchronisation quand les dépendances changent, puis lors du retrait du composant de l'écran. Ici cette fonction remet le titre antérieur ; elle ne produit aucune balise.

Chronologie avec un titre initial CDA

Moment	Action sur le titre
Afficher nom = Alice	Mémoriser CDA ; écrire Lecture de Alice.
Le parent passe nom = Bilal	Nettoyer Alice : remettre CDA.
Nouvel effet pour Bilal	Mémoriser CDA ; écrire Lecture de Bilal.
Retirer le composant	Nettoyer Bilal : remettre CDA.

Pourquoi nom doit être déclaré comme dépendance

L'effet lit `nom` : il doit suivre les nouveaux prénoms fournis par le parent. Avec `[]`, il conserverait la synchronisation initiale et ne suivrait pas le changement vers Bilal. Sans tableau, il serait exécuté après chaque rendu concerné, même si la valeur utile n'a pas changé. Avec `[nom]`, on annonce la valeur utilisée, et non un ordre arbitraire choisi pour éviter des exécutions.

Quelques limites utiles

Chaque ressource demande le nettoyage adapté : retirer un abonnement, arrêter un minuteur ou ignorer un résultat devenu périmé. Tous les effets n'ont pas nécessairement de nettoyage ; il est requis lorsque la synchronisation doit être annulée ou terminée. En développement, Strict Mode peut effectuer un cycle installation-nettoyage supplémentaire pour vérifier ce fonctionnement. Un effet bien nettoyé doit supporter ce test. Pour un simple calcul à partir des props, utiliser le rendu plutôt que fabriquer un état supplémentaire avec un effet.

À TOI DE JOUER · REA-04

Titre initial : CDA. Le composant ci-dessus reçoit `nom = "Alice"`, puis `"Bilal"`, puis est retiré. Quels titres attend-on après chaque synchronisation ou retrait ? Avec `[]`, quel changement serait manqué ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [39] [35] · Références détaillées à la fin.

5 · Attendre une donnée sans laisser un ancien résultat gagner

Objectif : distinguer chargement, succès et erreur, puis ignorer une réponse devenue périmée.

Prérequis : effet et nettoyage ; Promise, async et await expliqués dans TypeScript.

Une source fournie, sans API supposée

Une Promise représente un résultat futur. `resolve` fournit sa réussite ; `reject` son échec. Pour simuler une attente sans serveur personnel, la fonction `charger` ci-dessous utilise `setTimeout`, une fonction du navigateur qui appelle notre fonction après environ 300 millisecondes. Avec `id = "lire"`, le résultat est `Lire` ; tout autre `id` provoque une erreur. Cette source est fictive mais complète.

À placer hors du composant · la source est entièrement fournie

```
function charger(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id === 'lire') resolve('Lire');
      else reject(new Error('Activité inconnue'));
    }, 300);
  });
}
```

L'écran doit pouvoir dire ce qui se passe

Nous gardons un objet d'état avec `status` et `texte`. `status` vaut `"loading"` pendant l'attente, `"success"` à la réussite ou `"error"` à l'échec. Une liste réussie mais vide serait un succès demandant un message « Aucun résultat », pas forcément une erreur. Ici la source renvoie un seul texte, et l'état de départ doit indiquer le chargement. Les conditions du rendu choisissent un écran pour chaque état. Dans `Detail({ id })`, les accolades du paramètre extraient la prop `id` : c'est une écriture courte de `props.id`.

Fragment `Detail.jsx` · insérer l'effet au commentaire indiqué

```
import { useEffect, useState } from 'react';
export default function Detail({ id }) {
  const [etat, setEtat] = useState({
    status: 'loading', texte: ''
  });
  // Insérer ici le bloc useEffect ci-dessous.
  if (etat.status === 'loading') return <p>Chargement...</p>;
  if (etat.status === 'error') return <p>Erreur : {etat.texte}</p>;
  return <p>{etat.texte}</p>;
}
```

Pour approfondir : [39] [40] · Références détaillées à la fin.

SUITE DE LA LEÇON

5 · Attendre une donnée sans laisser un ancien résultat gagner

Préparer le lancement et son nettoyage

L'effet lit `id` et doit le déclarer comme dépendance. À chaque lancement, `actif` vaut `true`. Le nettoyage le passe à `false`. Une réponse ne doit mettre à jour l'état que si le lancement auquel elle appartient est toujours actif. Cela empêche une ancienne réponse de remplacer le résultat associé à un nouvel `id`.

Fragment · remplacer le commentaire dans `Detail` par cet effet

```
useEffect(() => {
  let actif = true;
  setEtat({ status: 'loading', texte: '' });
  async function lancer() {
    try {
      const texte = await charger(id);
      if (actif) setEtat({ status: 'success', texte });
    } catch (e) {
      if (actif) setEtat({ status: 'error', texte: e.message });
    }
  }
  lancer();
  return () => { actif = false; };
}, [id]);
```

Lire les lignes sensibles

`setEtat` annonce d'abord l'attente. `lancer` est une fonction `async` interne ; `await` y obtient le texte de la `Promise`. À la réussite, elle garde le texte ; au rejet, `catch` lit le message de l'`Error` construite par notre source. Le test `actif` existe dans les deux branches : une ancienne erreur ne doit pas non plus écraser une nouvelle réponse. L'effet lui-même retourne le nettoyage, pas la `Promise` de lancer.

Pour approfondir : [39] [40] · Références détaillées à la fin.

SUITE DE LA LEÇON

5 · Attendre une donnée sans laisser un ancien résultat gagner

Montage et limite de cette protection

Pour exécuter : placer `charger` hors de `Detail` dans `Detail.jsx`, remplacer le commentaire par l'effet, puis afficher `<Detail id="lire" />` depuis un parent dans l'environnement React décrit en leçon 1. Le chargement devient Lire ; avec un autre id, il devient Erreur : Activité inconnue. `actif` n'annule pas le minuteur ni une vraie requête : il ignore son résultat après nettoyage. Une vraie source peut aussi proposer un mécanisme d'annulation. C'est un exemple pour comprendre le cycle, pas une architecture complète de chargement.

À TOI DE JOUER · REA-05

Chronologie donnée : lancement `id="lire"` à 0 ms ; nouvelle prop `id="absent"` et deuxième lancement à 100 ms ; le premier résultat Lire arrive à 300 ms ; l'erreur du deuxième arrive à 400 ms. À 300 ms, l'ancien résultat peut-il remplacer le chargement actuel ? À 400 ms, quel état garde-t-on ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [39] [40] · Références détaillées à la fin.

6 · Une URL choisit l'écran et fournit un paramètre

Objectif : lire une route et suivre un segment d'URL jusqu'au composant. Prérequis : props et composants ; les exemples emploient le mode déclaratif de React Router.

Une route donne un sens à un chemin

Une URL identifie une adresse dans le navigateur. Dans `http://localhost:5173/activites/lire`, le chemin est `/activites/lire`. Un routeur rapproche ce chemin d'une règle pour choisir l'écran. React Router est une bibliothèque ajoutée à React pour ce rôle. `BrowserRouter` fournit le contexte lié à l'historique du navigateur ; `Routes` rassemble les règles ; chaque `Route` associe un path à un element, le composant à afficher. Ce n'est pas une route Express traitée par un serveur.

Le deux-points nomme une partie variable

Le motif `/activites/:id` possède une partie fixe, `activites`, puis une partie variable appelée `id`. `/activites/lire` correspond avec `id = "lire"` ; `/activites/marcher` avec `id = "marcher"`. `useParams` lit ces paramètres dans un composant rendu par le routeur. Il renvoie ici un objet dont la propriété `id` est un texte. Le nom `id` ne garantit pas un nombre : il faut convertir et vérifier si le contrat exige un identifiant numérique.

Règle complète pour Page

Dans cet exemple, seuls les mots `lire` et `marcher` sont admis. Le symbole `&&` demande que les deux comparaisons soient vraies : si `id` n'est ni `lire` ni `marcher`, on redirige vers `/activites/lire`. `Navigate` est un composant qui demande cette navigation ; `replace` remplace l'entrée actuelle de l'historique plutôt que d'ajouter une entrée pour le chemin refusé. Cette règle est une décision d'interface, pas une recherche dans une base cachée.

Fragment Page · imports et règle de cet exemple

```
import { useParams, Navigate } from 'react-router';
function Page() {
  const params = useParams();
  const id = params.id;
  if (id !== 'lire' && id !== 'marcher') {
    return <Navigate to="/activites/lire" replace />;
  }
  return <p>Activité : {id}</p>;
}
```

Pour approfondir : [42] [43] [41] [34] · Références détaillées à la fin.

SUITE DE LA LEÇON

6 · Une URL choisit l'écran et fournit un paramètre

Fragment App.jsx · avec Page et Navigate définis au-dessus

```
import { BrowserRouter, Routes, Route } from 'react-router';
export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/activites/:id" element={<Page />} />
        <Route path="*"
          element={<Navigate to="/activites/lire" replace />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Ce qu'il faut fournir en plus

Ces deux blocs appartiennent au même fichier App.jsx et requièrent react-router installé, en plus de l'environnement React de leçon 1. BrowserRouter est déjà placé dans App : ne pas ajouter un autre BrowserRouter autour dans cet exemple. La route "*" traite les autres chemins en redirigeant vers /activites/lire. La page initiale et la transformation JSX sont les seuls fichiers non fournis ici. Pour servir directement une URL imbriquée, le serveur du site doit aussi savoir renvoyer l'application ; le guide du routeur précise son installation.

Suivre des adresses complètes

Chemin demandé	Décision et affichage
/activites/lire	id = "lire" ; Activité : lire.
/activites/marcher	id = "marcher" ; Activité : marcher.
/activites/absent	Page refuse id, puis URL /activites/lire.
/autre	La route "*" redirige vers /activites/lire.

Pour approfondir : [42] [43] [41] [34] · Références détaillées à la fin.

SUITE DE LA LEÇON

6 · Une URL choisit l'écran et fournit un paramètre

Lire une URL ne donne pas un droit

Changer id dans l'adresse peut demander un autre écran, mais ne prouve pas que l'utilisateur a le droit de lire une donnée privée. Le serveur doit vérifier les droits de chaque requête. Une autre façon de transmettre le paramètre à un écran serait `<Detail id={id} />` : le routeur lit l'URL, puis le parent fournit une prop ordinaire. Cette séparation aide à distinguer navigation, données et permissions.

À TOI DE JOUER · REA-06

Motif fourni : `/activites/:id`. Avec `/activites/marcher`, quelle propriété lit `useParams` et quel est son type concret ? Avec `/activites/inconnue`, quelle règle de Page s'applique ? Modifier l'URL peut-il donner une permission serveur ?

Ouvrir le corrigé à la fin du cours.

Pour approfondir : [42] [43] [41] [34] · Références détaillées à la fin.

Corrigés des exercices

Compare les étapes du raisonnement, puis refais le cas en changeant une valeur. Chaque lien permet de revenir à la leçon.

MER-01 · 1. Comprendre un besoin avant de dessiner (exercice p. 7)

La demande est refusée. La date reste le 20 juin : le lecteur a déjà utilisé son unique renouvellement. Le critère comporte une situation précise et deux résultats observables : refus et date inchangée.

[Revenir à la leçon.](#)

MER-02 · 2. Passer des livres réels au MCD (exercice p. 9)

Oui pour le livre 8 : 0,n autorise zéro lien depuis un livre. Non pour la copie 12 : 1,1 impose exactement un livre depuis une copie. On lit la cardinalité proche de l'entité depuis une occurrence de cette entité.

[Revenir à la leçon.](#)

MER-03 · 3. Deux objets ou trois objets dans le même fait ? (exercice p. 11)

Bob-JavaScript-C2 échoue, car la paire Bob-JavaScript est absente. Alice-SQL-C2 passe les trois vérifications, mais n'existe pas dans les affectations initiales. Ce contre-exemple suffit à montrer que les paires ne conservent pas toute l'information.

[Revenir à la leçon.](#)

MER-04 · 4. Construire les tables : comprendre 1:n et n:n (exercice p. 14)

La ligne vaut commande_id = 101, produit_id = 6, quantite = 3. produit_id seul ne suffit pas : le produit 6 apparaît déjà dans la commande 100. Le couple (101, 6) distingue cette nouvelle ligne.

[Revenir à la leçon.](#)

MER-05 · 5. Du lien 1:1 aux contraintes du MPD (exercice p. 17)

(11, 1) est refusé : Alice possède déjà le dossier 10. (12, 3) est refusé : la personne 3 n'existe pas. (13, 2) est accepté : Bilal existe et n'a aucun dossier. Chaque test part du même état initial.

[Revenir à la leçon.](#)

MER-06 · 6. Dépendances, clés et 1NF : partir des données (exercice p. 20)

Ajouter (3, 0603) à TELEPHONE. La ligne du client existe déjà ; on ne la duplique pas. Un nouveau nom dans cet échantillon ne modifie pas la règle : les homonymes restent autorisés, donc nom ne devient pas une clé candidate.

[Revenir à la leçon.](#)

Corrigés · suite

MER-07 · 7. Normaliser avec des tables avant et après (exercice p. 24)

Non. produit → libellé reste vraie et le couple commande-produit reste une clé candidate. Ajouter une autre clé ne retire pas la dépendance partielle ni les modifications à plusieurs endroits. Il faut examiner toutes les candidates et déplacer le libellé dans PRODUIT.

[Revenir à la leçon.](#)

MER-08 · 8. Décrire un service avec UML : include et extend (exercice p. 27)

Utiliser include de Retirer vers Imprimer le reçu : le comportement est désormais inclus dans chaque retrait réussi. La flèche part du cas qui utilise le comportement et va vers le cas inclus.

[Revenir à la leçon.](#)

TYP-01 · 1 · Un type décrit les valeurs autorisées (exercice p. 29)

3 est un nombre et satisfait la règle. -1 est aussi un nombre : l'annotation number l'accepte, mais la règle métier le refuse. "3" est un texte : il est incompatible avec number dans le code vérifié. Pour une entrée extérieure, il faut contrôler son type, puis Number.isInteger et les bornes 1 à 5 pendant l'exécution. Le type seul n'exprime pas cette validation.

[Revenir à la leçon.](#)

TYP-02 · 2 · Décrire un objet, une liste et une position (exercice p. 31)

Seul ["A", 4] respecte les deux positions : texte puis nombre. [4, "A"] inverse leurs types ; ["A"] ne fournit pas la deuxième position obligatoire. pages vaut undefined, puisque la propriété facultative n'a pas été fournie. On doit traiter cette absence avant un calcul avec pages.

[Revenir à la leçon.](#)

TYP-03 · 3 · Vérifier une valeur inconnue avant de l'utiliser (exercice p. 33)

" Bilal " est un texte : trim donne Bilal. null n'est pas un texte : la fonction rend Texte requis. Dans la variante error, on lit message, donc Nom absent ; nom n'appartient pas à cette variante. as string ne fait qu'affirmer un type au vérificateur : il ne transforme ni ne vérifie la valeur pendant l'exécution.

[Revenir à la leçon.](#)

TYP-04 · 4 · Attendre un résultat et partager du code entre fichiers (exercice p. 36)

Les deux appels ont le type Promise<string> : ce type décrit le succès possible, sans garantir une réussite. await chargerNom(1) donne Alice. await chargerNom(9) conduit à catch avec Error("Utilisateur absent"). import { saluer } apporte la fonction exécutée ; import type apporte seulement une description retirée du JavaScript.

[Revenir à la leçon.](#)

Corrigés · suite

TYP-05 · 5 · Garder une relation de types avec un générique (exercice p. 39)

T est string, et l'élément en position 0 est Alice ; le type de retour général prévoit aussi undefined pour une autre liste qui serait vide. { id: 2 } convient : Partial autorise l'absence de nom et conserve id comme nombre. { nom: 2 } ne convient pas : nom est facultatif, mais doit être un texte quand il est fourni.

[Revenir à la leçon.](#)

SQL-01 · 1 · Une table conserve des faits et impose des règles (exercice p. 42)

Seul (12, 2, 0) est accepté : id 12 est nouveau, Bilal existe et le total 0 respecte ≥ 0 . Le premier ajout viole la référence : client 3 absent. Le deuxième viole CHECK : -1 est négatif. Le troisième viole NOT NULL sur client_id. L'absence de commande préalable de Bilal n'empêche pas une première commande.

[Revenir à la leçon.](#)

SQL-02 · 2 · Lire des lignes, puis filtrer, trier et limiter (exercice p. 45)

Le filtre garde les deux commandes. ASC place 20 avant 30 ; LIMIT 1 rend donc (11, 20). Avec OFFSET 1, on ignore d'abord cette première ligne, puis LIMIT 1 rend (10, 30). Le filtre n'a pas changé : seule la portion choisie dans le résultat trié a changé.

[Revenir à la leçon.](#)

SQL-03 · 3 · Joindre des tables et repérer une absence (exercice p. 48)

INNER JOIN produit deux lignes : Alice-10 et Alice-11. LEFT JOIN en produit trois : les deux précédentes et Bilal avec les colonnes de commande NULL. WHERE co.id IS NULL garde uniquement cette troisième ligne, donc Bilal. Une ligne sans correspondance ne représente pas une commande supplémentaire.

[Revenir à la leçon.](#)

SQL-04 · 4 · Compter et regrouper sans compter une absence (exercice p. 51)

1) COUNT(*) donne 1, car LEFT JOIN a conservé une ligne pour Bilal. 2) COUNT(co.id) donne 0, car l'id de commande de cette ligne est NULL. 3) Alice seule reste : elle a deux vrais id de commande et SUM vaut $30 + 20 = 50$. Le total affiché de Bilal serait 0 grâce à COALESCE, avant son exclusion par HAVING.

[Revenir à la leçon.](#)

Corrigés · suite

SQL-05 · 5 · Modifier des lignes et décider de valider ou annuler (exercice p. 54)

ROLLBACK annule le changement : 10 garde 30. COMMIT le conserverait : 10 aurait 99. Sans WHERE, l'UPDATE viserait les deux lignes existantes ; 10 et 11 seraient toutes deux à 99. Chaque décision se déduit du filtre, puis de la validation ou de l'annulation de la transaction.

[Revenir à la leçon.](#)

SQL-06 · 6 · Approfondissement : existence, paramètres et plan (exercice p. 57)

EXISTS garde Alice une seule fois : au moins une commande existe pour elle, aucune pour Bilal. La requête préparée avec 99 rend zéro ligne, car WHERE ne trouve aucune égalité. Le moteur peut ignorer un index s'il estime une autre méthode moins coûteuse. EXPLAIN ANALYZE exécute effectivement DELETE : ses effets doivent être pris en compte.

[Revenir à la leçon.](#)

DOC-01 · 1 · Une image devient un conteneur en cours d'exécution (exercice p. 60)

Aucun lien n'a été publié entre 8080 sur l'ordinateur et 3000 dans le conteneur. Ajouter EXPOSE ne créerait pas ce lien. Arrêter le conteneur ne détruit pas l'image ; start peut relancer ce conteneur.

[Revenir à la leçon.](#)

DOC-02 · 2 · Relier deux services et conserver les données (exercice p. 63)

L'outil SQL sur l'ordinateur ne peut plus utiliser localhost:15432. api peut toujours joindre db:5432 sur le réseau interne. down conserve records ; down -v le supprimerait.

[Revenir à la leçon.](#)

TES-01 · 1 · Vérifier une petite fonction avec un test unitaire (exercice p. 66)

1 → false, 2 → true, 8 → true, 9 → false. Les valeurs 2 et 8 sont les bornes admises ; 1 et 9 sont juste à l'extérieur. Ajouter aussi au moins une chaîne et un nombre décimal.

[Revenir à la leçon.](#)

TES-02 · 2 · Attendre une réponse HTTP et fermer ce qui a été ouvert (exercice p. 69)

Demander /tasks/99, attendre 404 puis { error: "Absente" }. Garder le nettoyage du serveur. C'est un test d'intégration entre client et serveur HTTP ; aucune base n'est utilisée dans cet exemple.

[Revenir à la leçon.](#)

BAC-01 · 1 · Le client demande, le serveur répond : HTTP (exercice p. 72)

404 selon notre contrat. Sans return, le traitement continuerait après le succès et tenterait d'écrire une seconde réponse ; une requête doit recevoir une réponse terminée de manière cohérente.

[Revenir à la leçon.](#)

Corrigés · suite

BAC-02 · 2 · Répartir le traitement : middleware, contrôleur, service (exercice p. 75)

400 : le décodeur JSON refuse la syntaxe avant d'atteindre une route, même si POST n'est pas une route prévue ici. Le repository porte le nouvel accès ; le contrôleur continue de choisir les réponses HTTP.

[Revenir à la leçon.](#)

BAC-03 · 3 · Valider une donnée reçue avant de l'utiliser (exercice p. 78)

Non : une annotation TypeScript ne contrôle pas la valeur reçue. `safeParse` renvoie `success:false` parce que "2" est un texte. Selon notre contrat, la réponse est 422 et aucune réservation n'est créée.

[Revenir à la leçon.](#)

BAC-04 · 4 · Identifier une personne puis vérifier son droit d'agir (exercice p. 81)

Répondre 403 et ne rien modifier. Le JWT établit ici l'identité 7 ; il ne lui donne pas la propriété du document. Le serveur compare cette identité au propriétaire 9 qu'il a lui-même enregistré.

[Revenir à la leçon.](#)

BAC-05 · 5 · Lire des données : SQL, ORM et changements cohérents (exercice p. 84)

`[{title:"Lire"}]` : Marcher est terminée, donc exclue, et seul title est demandé. Le titre de la tâche 1 reste Lire parce que le groupe d'écritures a été annulé.

[Revenir à la leçon.](#)

REA-01 · 1 · Un composant décrit l'écran avec des props (exercice p. 87)

Les deux paragraphes contiennent Prêt et Terminé : chaque parent fournit la prop `texte` à son instance du composant. Les accolades JSX insèrent le résultat de la lecture `props.texte` ; elles ne construisent pas un objet ici. La majuscule indique un composant défini en JavaScript, tandis qu'une balise en minuscules désigne un élément du navigateur.

[Revenir à la leçon.](#)

REA-02 · 2 · Un événement met à jour l'état, puis React rend à nouveau (exercice p. 90)

React traite les mises à jour successivement : la première transforme 0 en 1, la seconde transforme 1 en 2 ; l'affichage devient Compteur : 2. Chaque updater reçoit la valeur issue du précédent. Pour la saisie, `setNom("Bilal")` enregistre le texte ; `value` et le paragraphe liront le même état : Bilal et Bonjour Bilal.

[Revenir à la leçon.](#)

Corrigés · suite

REA-03 · 3 · Choisir le propriétaire de l'état et identifier les éléments (exercice p. 93)

Le parent modifie l'état grâce à `setNom`, que l'enfant a reçu sous le nom `onNom`. Il retransmet ensuite Bilal aux deux endroits : ils restent cohérents. Marcher garde key 20, même s'il devient le premier élément. Son identité n'a pas changé ; utiliser sa position aurait remplacé sa clé 1 par 0.

[Revenir à la leçon.](#)

REA-04 · 4 · Un effet synchronise une ressource extérieure au rendu (exercice p. 95)

Après la première synchronisation : Lecture de Alice. Au changement, le nettoyage remet CDA avant le nouvel effet, puis le titre devient Lecture de Bilal. Au retrait, le nettoyage remet CDA. Avec `[]`, le changement de `prop` ne relancerait pas l'effet : l'onglet resterait lié à Alice pendant que le paragraphe pourrait déjà afficher Bilal.

[Revenir à la leçon.](#)

REA-05 · 5 · Attendre une donnée sans laisser un ancien résultat gagner (exercice p. 98)

À 100 ms, le nettoyage passe l'actif du premier lancement à `false` ; le deuxième possède un autre actif, initialement `true`, et remet l'état en chargement. À 300 ms, le premier test if (actif) est faux : Lire est ignoré et le chargement reste affiché. À 400 ms, l'actif du deuxième est encore vrai : l'état devient `error` et affiche Activité inconnue. Le nettoyage protège le résultat actuel sans annuler physiquement la première opération.

[Revenir à la leçon.](#)

REA-06 · 6 · Une URL choisit l'écran et fournit un paramètre (exercice p. 101)

`useParams` fournit ici `params.id` égal au texte "marcher" ; le composant affiche Activité : marcher. Pour inconnue, `id` est différent de lire et de marcher : Page retourne `Navigate` et l'URL devient `/activites/lire`. La modification d'URL ne crée aucun droit ; le serveur doit vérifier l'identité et les permissions sur la ressource demandée.

[Revenir à la leçon.](#)

Lire les références

Les situations sont créées pour apprendre. Les liens ci-dessous donnent les références des notions : documentation des outils, spécification UML ou travaux académiques selon le sujet.

[01] Agile Alliance · Modèle de user story

Documentation officielle

<https://agilealliance.org/glossary/user-story-template/>

[02] OWASP · vérifier les permissions

Documentation officielle

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[03] OWASP · stockage des mots de passe

Documentation officielle

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[04] Sibel Adali, RPI · Dépendances, formes normales et décomposition

Référence académique

https://cs.rpi.edu/~sibel/csci4380/fall2020/course_notes/normalization.html

[05] MDN · Demandes et réponses HTTP

Documentation officielle

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>

[06] MDN · propriétés d'un cookie HTTP

Documentation officielle

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie>

[07] MDN · fonctions JavaScript

Documentation officielle

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions>

[08] MDN · variables et types JavaScript

Documentation officielle

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Grammar_and_types

[09] MDN · modules JavaScript

Documentation officielle

<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>

[10] MDN · transformer un tableau avec map

Documentation officielle

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map

[11] MDN · Fonctions async et résultats futurs

Documentation officielle

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

[12] Docker Compose · réseau et noms de services

Documentation officielle

<https://docs.docker.com/compose/how-tos/networking/>

Références · suite

[13] Docker Compose · démarrage et disponibilité

Documentation officielle

<https://docs.docker.com/compose/how-tos/startup-order/>

[14] Docker · volumes persistants

Documentation officielle

<https://docs.docker.com/engine/storage/volumes/>

[15] Docker · réutilisation du cache

Documentation officielle

<https://docs.docker.com/get-started/docker-concepts/building-images/using-the-build-cache/>

[16] Docker · définition d'un conteneur

Documentation officielle

<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

[17] Docker · définition d'une image

Documentation officielle

<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/>

[18] Docker Compose · arrêt et suppression

Documentation officielle

<https://docs.docker.com/reference/cli/docker/compose/down/>

[19] Docker · instructions du Dockerfile

Documentation officielle

<https://docs.docker.com/reference/dockerfile/>

[20] Express 5 · traitement des erreurs

Documentation officielle

<https://expressjs.com/en/guide/error-handling/>

[21] Express 5 · chaîne de middlewares

Documentation officielle

<https://expressjs.com/en/guide/using-middleware/>

[22] jsonwebtoken · signature, verify et decode

Documentation officielle

<https://github.com/auth0/node-jwt-token>

[23] Mocodo · Manuel de référence de l'auteur

Documentation officielle

https://laowantong.github.io/mocodo/doc/fr_refman.html

[24] Microsoft Learn · cycle Red, Green, Refactor

Documentation officielle

<https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/contact-manager/iteration-6-use-test-driven-development-cs>

[25] Microsoft Learn · Séparer les responsabilités en couches

Documentation officielle

<https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>

Références · suite

[26] Microsoft Learn · Rôle du repository dans l'accès aux données

Documentation officielle

<https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design>

[27] Microsoft Learn · organiser des tests unitaires avec Arrange, Act, Assert

Documentation officielle

<https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices>

[28] Pierre Gérard · Modélisation des systèmes d'information, cours universitaire

Référence académique

<https://lipn.univ-paris13.fr/~gerard/docs/cours/merise-slides.pdf>

[29] node-postgres · Requêtes paramétrées

Documentation officielle

<https://node-postgres.com/features/queries>

[30] Node.js · vérifier un résultat avec assert

Documentation officielle

<https://nodejs.org/api/assert.html>

[31] Node.js · créer un serveur HTTP

Documentation officielle

<https://nodejs.org/api/http.html>

[32] Node.js · exécuter des tests

Documentation officielle

<https://nodejs.org/api/test.html>

[33] Nanci et Espinasse · Merise, deuxième génération (ouvrage des auteurs)

Référence académique

<https://pageperso.lis-lab.fr/bernard.espinasse/wp-content/uploads/2021/12/LivreMerisePDF-total-12sept14.pdf>

[34] React · Créer une application

Documentation officielle

<https://react.dev/learn/build-a-react-app-from-scratch>

[35] React · Transmettre des props

Documentation officielle

<https://react.dev/learn/passing-props-to-a-component>

[36] React · Partager un état

Documentation officielle

<https://react.dev/learn/sharing-state-between-components>

[37] React · Mettre à jour un tableau

Documentation officielle

<https://react.dev/learn/updating-arrays-in-state>

Références · suite

[38] React · Champs input

Documentation officielle

<https://react.dev/reference/react-dom/components/input>

[39] React · useEffect et nettoyage

Documentation officielle

<https://react.dev/reference/react/useEffect>

[40] React · useState

Documentation officielle

<https://react.dev/reference/react/useState>

[41] React Router · Navigate

Documentation officielle

<https://reactrouter.com/api/components/Navigate>

[42] React Router · BrowserRouter

Documentation officielle

<https://reactrouter.com/api/declarative-routers/BrowserRouter>

[43] React Router · useParams

Documentation officielle

<https://reactrouter.com/api/hooks/useParams>

[44] Stéphane Crozat · Théorie de la normalisation relationnelle

Référence académique

<https://stph.scenari-community.org/bdd/nor1.pdf>

[45] tsx · Exécution et vérification des types

Documentation officielle

<https://tsx.is/typescript>

[46] Vitest · comparer les résultats

Documentation officielle

<https://vitest.dev/api/expect.html>

[47] OMG · UML 2.5.1, standard et document normatif, chapitre 18

Norme / spécification

<https://www.omg.org/spec/UML/2.5.1>

[48] PostgreSQL · Contraintes

Documentation officielle

<https://www.postgresql.org/docs/current/ddl-constraints.html>

[49] PostgreSQL · RETURNING

Documentation officielle

<https://www.postgresql.org/docs/current/dml-returning.html>

[50] PostgreSQL · Comparaisons et NULL

Documentation officielle

<https://www.postgresql.org/docs/current/functions-comparison.html>

Références · suite

[51] PostgreSQL · COALESCE et expressions conditionnelles

Documentation officielle

<https://www.postgresql.org/docs/current/functions-conditional.html>

[52] PostgreSQL · Sous-requêtes

Documentation officielle

<https://www.postgresql.org/docs/current/functions-subquery.html>

[53] PostgreSQL · Index

Documentation officielle

<https://www.postgresql.org/docs/current/indexes.html>

[54] PostgreSQL · LIMIT et OFFSET

Documentation officielle

<https://www.postgresql.org/docs/current/queries-limit.html>

[55] PostgreSQL · ORDER BY

Documentation officielle

<https://www.postgresql.org/docs/current/queries-order.html>

[56] PostgreSQL · PREPARE et paramètres

Documentation officielle

<https://www.postgresql.org/docs/current/sql-prepare.html>

[57] PostgreSQL · Agrégats

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-agg.html>

[58] PostgreSQL · DELETE

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-delete.html>

[59] PostgreSQL · Jointures

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-join.html>

[60] PostgreSQL · INSERT et données d'exemple

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-populate.html>

[61] PostgreSQL · Interroger une table

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-select.html>

[62] PostgreSQL · Transactions

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-transactions.html>

[63] PostgreSQL · UPDATE

Documentation officielle

<https://www.postgresql.org/docs/current/tutorial-update.html>

Références · suite

[64] PostgreSQL · EXPLAIN

Documentation officielle

<https://www.postgresql.org/docs/current/using-explain.html>

[65] Prisma ORM 6 · transactions

Documentation officielle

<https://www.prisma.io/docs/orm/v6/prisma-client/queries/transactions>

[66] Prisma ORM 6 · migrations en développement et déploiement

Documentation officielle

<https://www.prisma.io/docs/orm/v6/prisma-migrate/workflows/development-and-production>

[67] Prisma ORM 6 · méthodes du client

Documentation officielle

<https://www.prisma.io/docs/orm/v6/reference/prisma-client-reference>

[68] RFC 6750 · Défi d'authentification Bearer

Norme / spécification

<https://www.rfc-editor.org/rfc/rfc6750.html#section-3>

[69] RFC 9110 · Statut 401 et défi WWW-Authenticate

Norme / spécification

<https://www.rfc-editor.org/rfc/rfc9110.html#name-401-unauthorized>

[70] RFC 9110 · codes de réponse HTTP

Norme / spécification

<https://www.rfc-editor.org/rfc/rfc9110.html#name-status-codes>

[71] TypeScript · Bases et compilateur

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/basic-types.html>

[72] TypeScript · Types courants

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/everyday-types.html>

[73] TypeScript · Fonctions

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/functions.html>

[74] TypeScript · Génériques

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

[75] TypeScript · Modules et imports de types

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/modules.html>

[76] TypeScript · Narrowing et unions discriminées

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html>

Références · suite

[77] TypeScript · Types d'objets et tuples

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/2/objects.html>

[78] TypeScript · Types utilitaires

Documentation officielle

<https://www.typescriptlang.org/docs/handbook/utility-types.html>

[79] TypeScript · Référence tsconfig

Documentation officielle

<https://www.typescriptlang.org/tsconfig/>

[80] Zod · validation des données à l'exécution

Documentation officielle

<https://zod.dev/basics>